Bachelor of Science in Software Engineering
May 2026



AgentLint: Detecting Architectural Smells in LLM-Based Agent Systems

A Taxonomy of 14 Design Rules with Combined Static and Runtime Analysis

Adam Abdullah
Samra Krembi

Faculty of Computing, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden

This thesis is submitted to the Faculty of Computing at Blekinge Institute of Technology in partial fulfillment of the requirements for the degree of Bachelor of Science in Software Engineering. The thesis is equivalent to 10 weeks of full-time studies.

The authors declare that they are the sole authors of this thesis and that they have not used any sources other than those listed in the bibliography and identified as references. They further declare that they have not submitted this thesis at any other institution to obtain a degree.

Contact Information:
Author(s):
Adam Abdullah
E-mail: adad23@student.bth.se

Samra Krembi
E-mail: sakg23@student.bth.se

University advisor:
Dr. Usman Nasir
Department of Software Engineering

Faculty of Computing
Blekinge Institute of Technology
SE–371 79 Karlskrona, Sweden

Internet : www.bth.se
Phone : +46 455 38 50 00
Fax : +46 455 38 50 57

Abstract

Background. Large Language Model (LLM)-based agent systems are now widely used in software engineering, yet their non-deterministic reasoning, dynamic context management, and tool-based interaction introduce failure modes that traditional software quality approaches do not address. While code smells have been studied in conventional and machine learning systems, no prior work provides an architecture-level taxonomy of design issues specific to agentic AI architectures.
Objectives. This thesis aims to construct a taxonomy of architectural smells in LLM-based agent systems, develop a detection tool that implements the taxonomy through combined static and runtime analysis, and evaluate the combined effectiveness of static analysis and runtime instrumentation for detecting these smells.
Methods. The study employed a Design Science Research Methodology. The taxonomy was constructed by combining a literature review (2022–2026) with repository mining of 52 open-source agent frameworks covering 96,120 commits. The taxonomy was then implemented in AgentLint, a Python library implementing 14 design rules via AST-based static analysis and decorator-based runtime interception. The evaluation comprised a quantitative detection experiment across four agent frameworks (PraisonAI, PydanticAI, CrewAI, DeerFlow) with 200 runtime trials, and a qualitative developer evaluation with eight practitioners at an industrial site.
Results. Static analysis detected 279 findings spanning 11 of 14 rules across the four repositories, with missing tool-call error handling (DR-6) as the most prevalent smell (45.5%). Runtime analysis detected 268 smell instances across 200 trials covering 4 rules, confirming that code-level issues manifest as observable problems during live agent execution. The developer evaluation rated report clarity at 4.25/5 and location accuracy at 4.50/5, with practitioners confirming that the detected smell categories correspond to issues experienced in production.
Conclusions. Architectural smells in LLM-based agent systems can be classified and detected through combined static and runtime analysis. The taxonomy and tool provide a diagnostic framework that enables developers to identify reliability, maintainability, and cost-efficiency issues in agent-based systems before they reach production.

Keywords: architectural smells, LLM-based agents, static analysis, runtime instrumentation, design science

Acknowledgments

We would like to thank our academic supervisor, Usman Nasir, for his guidance and constructive feedback throughout this project. We are also grateful to Ricardo Britto, Emir Medic, and Nishrith Saini at Ericsson Karlskrona for making the industry collaboration possible, facilitating access to practitioners, providing valuable input on the evaluation design, and reviewing drafts of this thesis.

We thank the eight developers at Ericsson who participated in the evaluation. Their time and candid feedback shaped both the tool and this thesis.

We also wish to thank Torbjörn Fridensköld for his support during this work.

Finally, we thank our families for their constant support and for always standing by our side throughout this journey.

Use of AI Tools

The following AI tools were used during the development of this thesis:

- **Kiro** (Amazon): Used for code generation, code review, and iterative development of the AgentLint library. Kiro assisted with writing checker implementations, test scaffolding, and debugging during the design and development phase.
- **ChatGPT** (OpenAI): Used for brainstorming, literature search support, and drafting and revising thesis text. All AI-generated content was reviewed, verified, and edited by the authors.

In all cases, the authors take full responsibility for the final content. AI-generated outputs were treated as drafts and validated against primary sources before inclusion.

Contents

Abstract i

Acknowledgments iii

Use of AI Tools v

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 Problem Statement . . . 2
- 1.3 Research Questions . . . 2
- 1.4 Objectives and Contributions . . . 2
- 1.5 Scope and Delimitations . . . 3
- 1.6 Thesis Outline . . . 3

2 Related Work 5
- 2.1 Failure Analysis in LLM-Based Agent Systems . . . 5
- 2.2 Agent Architecture and Modular Design . . . 5
- 2.3 Code Smells in Software and ML Systems . . . 6
- 2.4 Combined Static and Runtime Analysis . . . 6
- 2.5 Prompt Security and Context Management . . . 6
- 2.6 State Management in Concurrent Agent Systems . . . 7
- 2.7 Research Gap and Positioning . . . 7

3 Research Design 9
- 3.1 Design Science Research Process . . . 9
  - 3.1.1 Phase 1: Problem Identification . . . 9
  - 3.1.2 Phase 2: Definition of Objectives . . . 13
  - 3.1.3 Phase 3: Design and Development . . . 13
  - 3.1.4 Phase 4: Demonstration . . . 16
  - 3.1.5 Phase 5: Evaluation . . . 18
  - 3.1.6 Phase 6: Communication . . . 20
- 3.2 Validity and Reliability . . . 20

4 Results and Analysis 21
- 4.1 Step 1: Static Detection Capability . . . 21
- 4.2 Step 2: Runtime Detection Capability . . . 23
- 4.3 Step 3: Static vs. Runtime Comparison . . . 25
- 4.4 Developer Evaluation Survey . . . 26

5 Discussion **31**
5.1 Answering the Research Questions 31
5.2 Framework-Specific Observations 32
5.3 Unexpected Findings 33
5.4 Configuration Sensitivity 34
5.5 Checker Limitations 34
5.6 Implications for Practice 35
5.7 Ethical and Societal Implications 35
5.8 Relation to Existing Literature 35
5.9 Limitations and Threats to Validity 36

6 Conclusions and Future Work **39**

References **41**

A Developer Evaluation Questionnaire **43**

B Full Taxonomy of Design Rules **47**

List of Figures

3.1 Design Science Research Process (adapted from [11]) 9
3.2 Distribution of architectural smell hits by category 11
3.3 Repository selection and mining pipeline 12
3.4 C4-level container diagram of the AgentLint library 16
3.5 Sequence diagram of the static analysis pipeline 16
3.6 Runtime analysis flow: the interceptor captures LLM and tool-call events after completion, routes them to checkers, and generates a report at session end 17
3.7 Candidate selection pipeline for the evaluation phase 18
3.8 Timeline of the unified runtime test scenario. Phase 1 grows context through multi-turn tool use; Phase 2 triggers failure-mode smells. . . 19

4.1 Distribution of smell counts per trial (50 trials per repository) 24
4.2 Venn diagram of design rule detection coverage. Static detects 11 rules, runtime 4, with 4 in overlap. 25
4.3 Q25: smell categories participants have experienced in production (multi-select, $n = 8$). 28

List of Tables

2.1 Comparison of related works along key analytical dimensions 8
2.2 Mapping of taxonomy categories to supporting literature 8

3.1 Mapping from literature themes to repository mining keywords 10
3.2 Repository mining summary statistics 11
3.3 Architectural Smell Categories and Design Rules (see Appendix B for full descriptions) 14
3.4 Detection coverage of AgentLint across analysis modes 17

4.1 Static analysis findings per repository and design rule 21
4.2 Static findings by severity and confidence per repository 22
4.3 Runtime analysis results per repository (50 trials each) 23
4.4 Design rule detection coverage: static vs. runtime analysis 25
4.5 Participant background ($n = 8$) 26
4.6 Participant familiarity and tool usage ($n = 8$) 26
4.7 Likert-scale results for usefulness and usability (Q17–Q20), $n = 8$, scale 1–5 27
4.8 Design rules ranked by usefulness ($n = 8$) 28
4.9 Design rules ranked by noisiness ($n = 8$) 28

B.1 Full taxonomy of 14 architectural design rules 47

Chapter 1

Introduction

1.1 Background

Large Language Models (LLMs) have evolved from text-generation systems into the core of autonomous software agents. These agents combine language understanding with tool use, memory, and multi-step planning to perform complex tasks with little human input [17, 19]. Frameworks such as LangChain, CrewAI, AutoGPT, and PydanticAI let developers build agent applications for software engineering, research, and enterprise workflows.

However, the same features that make these agents powerful also create new failure modes. Traditional software has predictable control flow and reproducible errors. Agent systems instead rely on non-deterministic reasoning, dynamic context windows, and tool-based interactions that produce complex, hard-to-predict failures [8]. An agent may loop forever consuming tokens, lose its persona after a tool call, or corrupt shared state when multiple workers run at the same time. Standard static analysis tools and code smell taxonomies do not cover these issues.

The concept of *code smells* (patterns that indicate potential quality problems) has been used to find maintainability issues in traditional software [22] and in machine learning systems [6]. However, no one has applied this concept to LLM-based agent architectures. The challenges specific to agents, such as unpredictable behaviour, prompt-based control, and the interaction between code and LLM behaviour, call for a domain-specific approach.

In this thesis, an *architectural smell* is a recurring design pattern in an LLM-based agent system that signals a potential quality problem without being a bug itself. Examples include poor reliability, wasted tokens, or security weaknesses. The concept extends code smells [22] from functions or classes to the architecture level, focusing on how an agent's reasoning, tools, memory, and orchestration interact.

The practical relevance of this problem is supported by empirical evidence. Islam et al. [8] analysed 1,187 bug-related posts across seven agent frameworks and found that unstable reasoning chains and execution failures account for a substantial share of reported defects. Ehsani et al. [2] studied over 33,000 agent-authored pull requests and identified systematic CI/CD rejection patterns tied to architectural shortcomings. Hasan et al. [7] examined 856 tools across 103 MCP servers and reported that 97.1% contain at least one quality smell. These findings indicate that architectural weaknesses in agent systems are widespread, costly, and not addressed by existing tooling.

1.2 Problem Statement

Research on LLM agent failures exists but remains scattered. Studies tend to focus on one aspect at a time, such as prompting strategies [3], reasoning stability [1, 15], or tool-use efficiency [7], without connecting these issues to their shared architectural roots. No existing taxonomy covers recurring design issues at the architecture level of LLM-based agent systems, and no tool combines code-level analysis with runtime observation to detect them. Without such resources, developers discover these problems only through production failures or manual review.

This thesis addresses this gap by proposing a taxonomy of architectural smells in LLM-based agent systems and developing AgentLint, a Python library that detects these smells through combined static and runtime analysis. The study targets Python-based agent systems and focuses on architecture-level design issues rather than prompt quality or model behaviour.

1.3 Research Questions

The study is guided by three research questions:

- **RQ1:** What recurring architectural smells can be identified in LLM-based agent systems, and how can they be categorised into a taxonomy?
- **RQ2:** How can these architectural smells be detected through combined static and runtime analysis of Python-based agent code?
- **RQ3:** How effective is runtime instrumentation at detecting architectural smells compared to static analysis alone?

RQ1 asks what goes wrong at the architecture level and how these issues can be grouped. RQ2 asks how the taxonomy can be turned into an automated detection tool. RQ3 asks whether runtime analysis catches smells that static analysis misses, since some issues only appear when the agent is actually running. RQ3 is deliberately comparative and more broadly applicable than RQ1 and RQ2: while RQ1 and RQ2 are specific to the agent domain, RQ3 contributes to the general understanding of when runtime analysis adds value over static analysis alone, with findings potentially transferable to other non-deterministic software domains.

1.4 Objectives and Contributions

The thesis makes three contributions:

1. **A taxonomy of 14 architectural design rules** for LLM-based agent systems, grouped into six categories (reasoning loops, memory and context bloating, persona and tool failures, error recovery and resilience, prompt and instruction drift, and state management). The taxonomy was built by combining a literature review with repository mining of 52 open-source agent frameworks covering 96,120 commits.

2. **AgentLint**, a Python library that implements all 14 design rules through two analysis modes: Abstract Syntax Tree (AST) based static analysis for scanning code offline, and decorator-based runtime analysis for monitoring live agents.

3. **An empirical evaluation** that tests AgentLint on four agent frameworks (PraisonAI, PydanticAI, CrewAI, DeerFlow) with 200 runtime trials, and collects feedback from eight practitioners at an industrial site. The evaluation measures which smells each analysis mode detects and how practitioners rate the tool's output.

1.5 Scope and Delimitations

This section consolidates the boundaries of the study to set clear expectations for the reader.

Language and ecosystem. The analysis is restricted to Python-based agent frameworks. Python was chosen because it is the dominant language for LLM agent frameworks (LangChain, CrewAI, AutoGPT, PydanticAI are all Python-based) and its AST introspection and decorator mechanisms enable AgentLint's static and runtime analysis. Other languages and ecosystems are not considered.

Abstraction level. The work targets architecture-level structural patterns, specifically how components are composed and configured. Prompt engineering quality, model behaviour, and output correctness are outside the scope of this thesis.

Frameworks evaluated. The runtime evaluation covers four frameworks: CrewAI, PraisonAI, PydanticAI, and DeerFlow. These were selected based on the criteria described in Section 3.1.4. Other frameworks may exhibit additional or different smells not captured here.

Model configuration. All runtime experiments use a single LLM configuration: `gemma3:27b` with a context window of 8192 tokens, served locally through Ollama. Results may differ under other models or context sizes.

Evaluation metrics. Because no established ground-truth dataset of architectural smells in agent systems exists, formal precision and recall figures cannot be reported. The evaluation instead relies on manual inspection and qualitative assessment, as discussed in Section 5.9.

Tool availability. AgentLint was developed at Ericsson and must undergo their intellectual property registration process before it can be released as open-source software. As a result, the tool is not publicly available at the time of submission.

1.6 Thesis Outline

- **Chapter 2: Related Work** reviews prior work on LLM agent failures, agent architecture, code smells in ML systems, prompt security, and state management. It identifies the research gap and maps existing literature to the six taxonomy categories.

- **Chapter 3: Research Design** describes the Design Science Research Methodology used in this study, covering the six-phase process, repository mining,

taxonomy construction, and the evaluation design.

- **Chapter 4: Results and Analysis** presents findings from three evaluation steps: static detection (Step 1), runtime detection (Step 2), and the comparison between the two modes (Step 3). It also reports the developer evaluation results.
- **Chapter 5: Discussion** interprets the results against the research questions, discusses framework-specific observations, configuration sensitivity, checker limitations, and threats to validity.
- **Chapter 6: Conclusions and Future Work** summarises the contributions, answers the three research questions, and outlines future work directions.

Chapter 2

Related Work

This chapter reviews prior work relevant to architectural smells in LLM-based agent systems and identifies the research gap. The review progresses from the problem space (Section 2.1: what fails in agent systems) through architectural foundations (Section 2.2) and existing detection approaches (Sections 2.3–2.4) to the specific domains where smells manifest (Sections 2.5–2.6). Each section identifies what existing work contributes and what gap remains, building toward the research gap analysis in Section 2.7.

2.1 Failure Analysis in LLM-Based Agent Systems

Islam et al. [8] analysed 1,187 bug-related posts across seven LLM agent frameworks, identifying recurring failure patterns such as unstable reasoning chains, execution failures, and tool-use errors. Their work provides the most comprehensive empirical catalogue of what goes wrong in agent systems, but does not trace these failures to their architectural causes or propose automated detection. Ning et al. [9] propose a defect taxonomy and develop Agentable, a static analysis tool using Code Property Graphs (a program representation that combines AST, control flow, and data flow into a single queryable graph). Agentable can identify structural patterns in agent code but cannot detect issues that appear only at runtime, such as context window overflow or retry storms that depend on LLM behaviour. Ehsani et al. [2] studied 33,000 agent-authored pull requests and found that non-merged PRs tend to fail CI/CD validation due to weaknesses in error recovery, providing evidence that architectural quality directly affects agent output acceptance.

Together, these studies show that failures in LLM agent systems are widespread and costly, but focus on behavioural symptoms rather than structural causes. This thesis shifts focus to the recurring architectural patterns that produce these symptoms and provides automated detection for them.

2.2 Agent Architecture and Modular Design

Wang et al. [17] decompose LLM-based agents into profiling, memory, planning, and action modules. Xi et al. [19] survey single-agent and multi-agent scenarios, discussing coordination mechanisms relevant to state management smells. Shang et al. [12] formalise the modular design space into four modules and show that poor configurations significantly hurt task success. This thesis builds on their decompo-

sition to organise smells into categories that map to architectural modules, adding the quality assessment dimension these works lack.

2.3 Code Smells in Software and ML Systems

Zhang et al. [22] identified 22 ML-specific code smells targeting conventional ML pipelines. Britto et al. [6] developed MLpylint using Design Science Research and evaluated it with 15 professionals, making it the closest methodological precedent to this thesis, though limited to ML pipeline code. Wu et al. [18] propose iSMELL, combining LLMs with expert toolsets, showing that multiple detection strategies outperform single-mode analysis. No prior work addresses architectural smells specific to agentic systems.

2.4 Combined Static and Runtime Analysis

Hajdu et al. [5] report a 76% improvement in detected bugs when combining static and dynamic analysis in WhatsApp's Erlang codebase. Their key insight is that static analysis catches structural issues (dead code, type mismatches) while dynamic analysis catches behavioural issues (race conditions, resource leaks) that only manifest during execution. This complementarity motivates the dual-mode approach in this thesis. OpenLLMetry [16] provides observability for generative AI applications based on OpenTelemetry, demonstrating that non-intrusive runtime instrumentation of LLM calls is technically feasible without modifying application code. Souza et al. [13] propose PROV-AGENT for provenance tracking in agentic workflows, enabling post-hoc analysis of agent decision chains. None of these apply combined analysis to detecting architectural smells in LLM-based agent systems, leaving a gap between general-purpose analysis tools and agent-specific quality concerns.

2.5 Prompt Security and Context Management

Greshake et al. [3] introduce indirect prompt injection, showing that adversaries can exploit LLM-integrated applications by embedding malicious instructions in external data sources (websites, documents) that the agent retrieves and processes. This establishes unsanitised external input as a structural vulnerability in any agent that processes retrieved content, motivating DR-10 (Input Isolation) in the taxonomy. Chen et al. [1] argue that agents without persistent state across reasoning steps are susceptible to instruction drift, where the original task intent degrades over multiple turns. Hasan et al. [7] examined 856 tools across 103 Model Context Protocol (MCP) servers and found that 97.1% of tool descriptions contain at least one smell (ambiguous parameters, missing error documentation, or inconsistent schemas), showing that the tool ecosystem itself introduces quality risks. Yadav et al. [20] formalise Agentic Retrieval-Augmented Generation (RAG) as Partially Observable Markov Decision Processes (POMDPs) and show that unbounded context accumulation degrades reasoning quality proportionally to context length, providing theoretical grounding for

DR-3 (Context Bloat). Suri et al. [15] demonstrate that reasoning instability correlates with non-converging trajectories, achieving 20% improvement through query refinement strategies.

2.6 State Management in Concurrent Agent Systems

Ehsani et al. [2] document that CI failures in agent-authored code often stem from state corruption during concurrent execution, where multiple agents or threads modify shared resources without synchronisation. Xi et al. [19] discuss coordination mechanisms for multi-agent systems (message passing, shared blackboards, hierarchical delegation) but do not identify recurring anti-patterns that arise when these mechanisms are implemented incorrectly. The repository mining in this thesis extends these findings with empirical evidence from 52 frameworks, where unsynchronised shared state (DR-12), non-deterministic task ordering (DR-14), and unsafe serialisation of agent state (DR-13) were observed as recurring patterns across multiple independent projects.

2.7 Research Gap and Positioning

The preceding sections show that existing work addresses individual aspects of LLM agent quality (code smells, runtime observability, prompt security, state management) but no prior study combines these into a unified architectural perspective with automated detection. Table 2.1 compares the most related works along key analytical dimensions.
This thesis is the first to combine all six dimensions. Table 2.2 maps the taxonomy categories to supporting literature.

Table 2.1: Comparison of related works along key analytical dimensions

Work	Taxonomy	Tool	Arch.-level	Static+Runtime	Pract. eval.	Agent-specific
Zhang et al. [22]	✓	—	—	—	—	—
Britto et al. [6]	✓	✓	—	Static only	—	—
Wu et al. [18]	—	✓	—	Static only	—	—
Ning et al. [9]	✓	✓	Partial	Static only	—	✓
Hajdu et al. [5]	—	✓	—	✓	—	—
Islam et al. [8]	—	—	—	—	—	✓
Ehsani et al. [2]	✓	—	—	—	—	✓
This thesis	✓	✓	✓	✓	✓	✓

Table 2.2: Mapping of taxonomy categories to supporting literature

Category	References	Focus Area
Reasoning Loops	[15], [8], [9]	Non-converging trajectories, unstable planning
Memory/Context Bloating	[20], [1], [13]	Unbounded context growth, hallucination propagation
Persona/Tool Failures	[7], [12]	Incomplete tool descriptions, persona drift
Error Recovery	[2], [8]	PR rejection patterns, CI failures
Prompt/Instruction Drift	[3], [1], [7]	Prompt injection, ambiguous tool descriptions
State Management	[2], [19]	State corruption, coordination anti-patterns

Chapter 3

Research Design

This study employs the Design Science Research Methodology (DSRM) [10, 11]. DSRM was selected because its phases map directly to the three research questions: problem identification (RQ1), artifact construction (RQ2), and structured evaluation (RQ3). The process follows the six-phase model proposed by Peffers et al. [11].

Before selecting Design Science, three alternatives were considered. Case Study Research [21] could show how issues appear in specific systems but does not prescribe artifact construction (RQ2) or quantitative comparison (RQ3). Grounded Theory [14] could support taxonomy building (RQ1) but lacks artifact development and evaluation phases. A Systematic Literature Review could partially address RQ1 but cannot produce a tool or a static-versus-runtime comparison. Design Science is the best fit because it combines problem understanding (Phase 1, RQ1) with artifact construction (Phase 3, RQ2) and structured evaluation (Phase 5, RQ3).

3.1 Design Science Research Process

The research process follows the six-phase DSRM model proposed by Peffers et al. [11]: problem identification, definition of objectives, design and development, demonstration, evaluation, and communication. The complementary Offermann et al. formulation [10] informed the validation activities in Phases 4 and 5. Figure 3.1 shows the six phases.

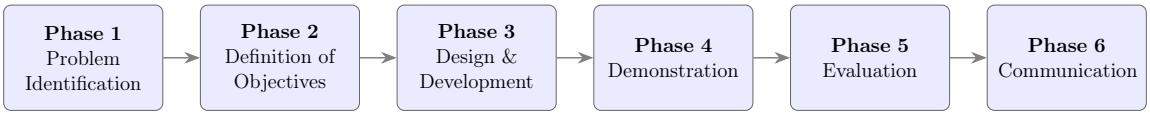


Figure 3.1: Design Science Research Process (adapted from [11])

The research method follows the Design Science process described above. The following subsections detail each phase. Throughout this thesis, three terms distinguish detection contexts: a *smell hit* refers to a commit-level match during repository mining, a *finding* refers to a static analysis detection in source code, and an *instance* refers to a runtime detection during agent execution.

3.1.1 Phase 1: Problem Identification

The problem described in Section 1.2 was investigated through a combination of literature review and repository mining.

Literature Review The literature review used snowball sampling starting from three seed papers identified through Google Scholar searches for "LLM agent failures," "code smells machine learning," and "agent architecture quality." From these seeds, forward and backward citation tracking was applied to identify related work. Additional searches were conducted on arXiv for recent preprints in the rapidly evolving agent domain. The review covered research published between 2022 and 2026 and focused on LLM agents, failure patterns, agent architecture, and software engineering challenges. Inclusion criteria were: English language, empirical or tool-based contribution, and focus on LLM-based agent systems or code/architecture quality. Exclusion criteria were: purely theoretical work without empirical grounding, studies targeting traditional (non-LLM) software exclusively, and duplicate preprint/publication pairs (where the published version was preferred). Approximately 85 papers were screened by title and abstract; 22 were included in the final review and are cited in Chapter 2.

The literature review identified six recurring failure themes that informed the repository mining keywords: (1) non-converging reasoning and infinite loops, (2) unbounded context growth and memory overflow, (3) tool interaction failures and persona drift, (4) missing error recovery and retry logic, (5) prompt injection and instruction degradation, and (6) state corruption in concurrent execution. Table 3.1 shows how these themes were operationalised into keyword sets for the mining phase.

Table 3.1: Mapping from literature themes to repository mining keywords

Literature Theme	**Mining Keywords**
Reasoning Loops	`loop`, `infinite`, `max_iterations`, `recursion`, `stuck`, `converge`
Memory/Context Bloating	`context`, `token`, `overflow`, `truncat`, `history`, `memory`, `append`
Persona/Tool Failures	`persona`, `system_prompt`, `tool_call`, `function_call`, `schema`
Error Recovery	`retry`, `backoff`, `fallback`, `timeout`, `exponential`, `error_handling`
Prompt/Instruction Drift	`inject`, `sanitiz`, `template`, `f-string`, `format`, `mutable`
State Management	`lock`, `mutex`, `thread`, `async`, `shared_state`, `race`, `serial`

Repository Mining Repository mining was conducted on 52 open-source LLM agent repositories, including frameworks such as LangChain, CrewAI, and PraisonAI. The mining process worked as follows: for each repository, all commits were scanned using keyword-based and structural pattern matching to identify changes related to architectural smell categories (e.g., error handling additions, context management changes, retry logic modifications). Each matching commit was recorded as a *smell hit* and assigned to one of the six taxonomy categories. The analysis covered 96,120 commits and identified 873 architectural smell hits—commit-level occurrences where

a code change matches one or more architectural smell patterns—of which 596 were classified as high-relevance (relevance score ≥ 5). Smells were detected in 36 of the 52 repositories. The following summary statistics and distribution are included here to demonstrate that the mining process produced sufficient data across all six categories to proceed to taxonomy construction; the detailed analysis of detection results is presented in Chapter 4. Table 3.2 summarises the key statistics from the mining process, and Figure 3.2 shows the distribution of smell hits across the six taxonomy categories.

Table 3.2: Repository mining summary statistics

Metric	Value
Total repositories analysed	52
Repositories with smells detected	36
Repositories without smells	16
Total commits scanned	96,120
Total smell hits (all relevance scores)	873
High-relevance hits (score ≥ 5)	596
Repositories with high-relevance hits	34

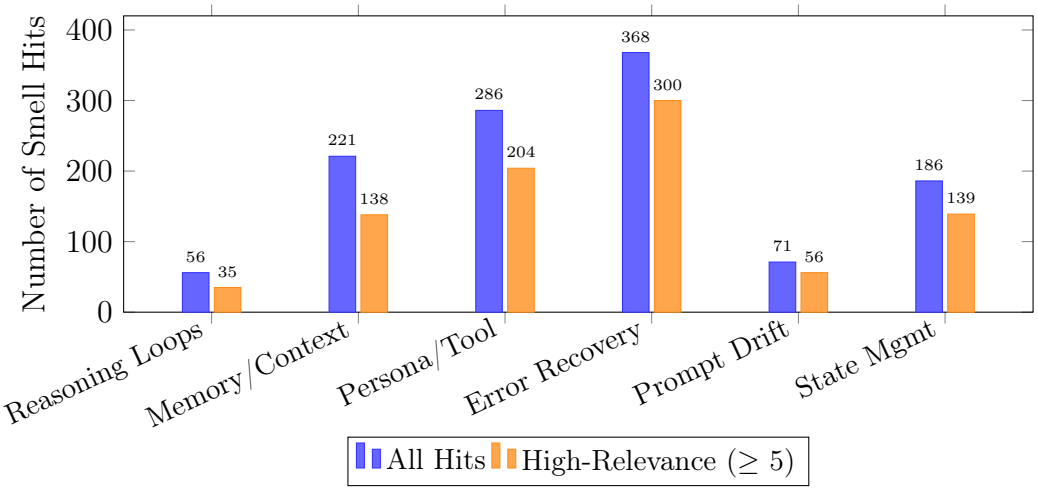


Figure 3.2: Distribution of architectural smell hits by category

Repository Selection Criteria: Candidate repositories were identified through three discovery methods: GitHub search using keywords such as "LLM agent", "AI agent framework", and "autonomous agent"; curated community lists (e.g., awesome-llm-agents); and snowball sampling from framework documentation and dependency graphs. This yielded approximately 100 candidates.

Inclusion and exclusion criteria were then applied to filter this pool down to 52 repositories. The selection prioritised diversity of architectural style (single-agent, multi-agent, graph-based, crew-based) to ensure the taxonomy captures patterns across different design approaches. Figure 3.3 shows the full pipeline from discovery to taxonomy construction, including filtering criteria at each step.

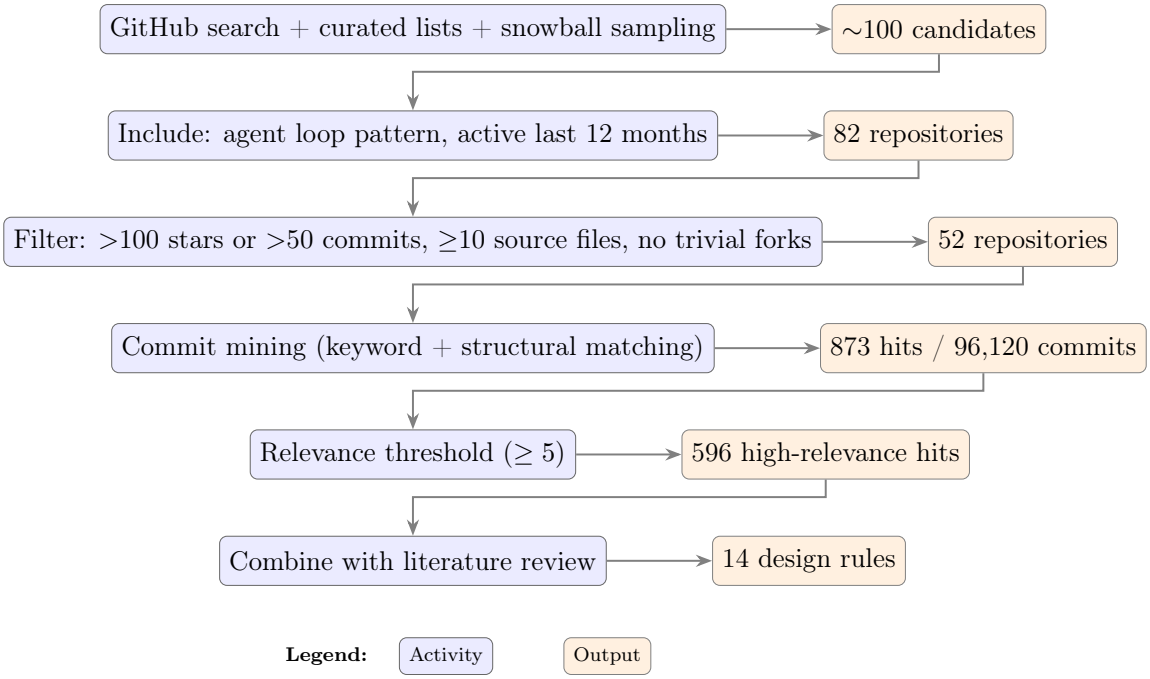


Figure 3.3: Repository selection and mining pipeline

Relevance Scoring Each commit identified during the repository mining was assigned a relevance score from 1 to 10. The score is composed of three sub-scores:

1. **Keyword density (0–4 points):** the number of distinct keywords from the same smell category that appear in the commit message or diff. A single keyword yields 1 point; four or more distinct keywords yield the maximum of 4.

2. **Code location (0–2 points):** commits modifying production source files (e.g., `src/`, `lib/`) receive 2 points; commits affecting only test or documentation files receive 0.

3. **Agent-core proximity (0–4 points):** commits touching directories associated with agent orchestration, LLM integration, or core control logic (e.g., paths containing `agent`, `llm`, `core`, `orchestrat`) receive up to 4 points based on the number of such paths modified.

The three sub-scores are summed to produce the final relevance score (maximum 10). A manual review of 10% of hits per category was conducted by both authors independently, with disagreements resolved through discussion. The sample was selected using purposive sampling across the score distribution: commits from the top, bottom, and middle of the ranked list were reviewed. Emphasis was placed on commits with high keyword density but low overall scores to understand what caused low rankings despite strong keyword matches. This calibrated the threshold. Scores ≥ 5 (moderate to high confidence) were retained, yielding 596 of 873 hits (68.3%). The threshold of 5 was chosen because hits below this score typically matched only a single keyword without structural evidence in the commit diff, indicating superficial matches rather than genuine architectural smell occurrences.

3.1.2 Phase 2: Definition of Objectives

The objective was defined as developing a taxonomy of architectural smells specific to LLM-based agent systems and implementing AgentLint, a static and runtime analysis tool, for their automated detection. The taxonomy should be empirically grounded (literature and mining), and the tool should demonstrate that the identified smells are detectable in real frameworks. These objectives map directly to the research questions: the taxonomy addresses RQ1, the tool addresses RQ2, and the comparison of static versus runtime detection addresses RQ3.

3.1.3 Phase 3: Design and Development

The taxonomy was built by combining evidence from literature and repository mining. During the literature phase, traditional architecture smell catalogues were reviewed, covering patterns such as cyclic dependencies, god components, and unstable interfaces; however, these target structural coupling concerns that do not map to the failure modes specific to LLM-based agent systems and were therefore not adopted. Instead, three of the six categories (Reasoning Loops, Memory/Context Bloating, Persona/Tool Failures) were derived from prior work that directly documents LLM agent failures, while the remaining three (Error Recovery/Resilience, Prompt/Instruction Drift, State Management Issues) were inspired by concerns raised in literature and subsequently confirmed through mining. The six categories are partially informed by [12]'s modular decomposition, which provided a structural frame for where smells manifest within an agent's architecture.

Regarding the role of commits: literature provided the initial category structure based on documented failure types, while repository mining provided empirical confirmation that these categories manifest as recurring fix patterns in real codebases. Mining further informed the specific detection targets for each design rule. For example, the frequency of retry-related commits that introduced backoff logic informed the 2-second threshold in DR-8 (Bounded Retry). Each design rule was included only if supported by at least one source (literature or mining); rules supported by both sources have stronger empirical justification. Specifically, a candidate rule required either (a) documentation in at least one peer-reviewed paper describing the failure mode, or (b) occurrence in at least three independent repositories during mining. Of the final 14 rules, 9 are supported by both literature and mining, 3 by literature only, and 2 by mining only. The final taxonomy contains six categories with 14 design rules (DR-1 through DR-14). The term "design rule" was chosen over "architecture smell" because each entry prescribes a specific structural property that should hold (e.g., "loops must have a bounded iteration count"). A violation of a design rule constitutes an architectural smell. This framing follows the convention in static analysis tooling where rules define what to check and violations are the detected issues. Table 3.3 summarises the categories.

DR-7 (Parallel Isolation) and DR-12 (Synchronized State) share similar detection logic under *State Management Issues*, differing only in target: DR-7 flags unsynchronised tool/agent resources across threads, while DR-12 flags unsynchronised agent-state containers.

The *Prompt/Instruction Drift* category does not analyse prompt content or string

Table 3.3: Architectural Smell Categories and Design Rules (see Appendix B for full descriptions)

Category	**Rules**	**Description**
Reasoning Loops	DR-1, DR-2	Unbounded loops, repeated identical tool calls
Memory/Context Bloating	DR-3, DR-4	Unbounded message history, unfiltered tool output
Persona/Tool Failures	DR-5, DR-6	Persona drift, missing tool-call error handling
Error Recovery/Resilience	DR-8, DR-9	Retry without backoff, missing fallback paths
Prompt/Instruction Drift	DR-10, DR-11	Unsanitised input in prompts, mutable templates
State Management Issues	DR-7, DR-12–14	Parallel isolation, unsynchronised state, serialization, nondeterministic ordering

quality. DR-10 (Input Isolation) checks whether user-supplied values are interpolated into system-level prompt strings without sanitisation. DR-11 (Immutable Templates) checks whether prompt templates are reassigned or mutated after definition. DR-5 (Persona Pinning) checks whether the system prompt is preserved across tool-call boundaries rather than being lost during multi-turn execution. All three target how prompts are *handled in code* (passed, preserved, constructed), not what they say, and are detectable through AST analysis.

Checker Implementation Example To illustrate how a static checker operates, Listing 3.1 shows the core logic of the DR-3 (Context Window) checker, which detects unbounded message history growth.

Listing 3.1: Simplified core logic of the DR-3 static checker

```python
class ContextBloatChecker(StaticChecker):
    """DR-3: Detect unbounded message history growth."""

    def visit_ast(self, tree, filepath, config):
        for node in ast.walk(tree):
            if isinstance(node, (ast.While, ast.For)):
                for child in ast.walk(node):
                    if self._is_append_call(child, config.
                        keywords):
                        if not self._has_size_guard(node):
                            yield SmellResult(
                                rule="DR-3",
                                severity="warning",
                                file=filepath,
                                line=child.lineno,
                                message="Unbounded append in
                                    loop"
                            )
```

The checker walks the AST, identifies loops containing `append()` calls on message-like objects, and verifies whether a size guard exists. All 14 static checkers follow this pattern.

AgentLint Architecture and Correctness AgentLint's architecture is based on four decisions: a *pluggable checker pattern* with auto-discovery from `checkers/`; a shared interface with `StaticChecker` and `RuntimeChecker` subclasses; a *fail-open runtime strategy*, meaning the runtime observer never blocks or disrupts the monitored agent even if a checker fails; and `astroid`-based type inference with heuristic fallback.

Figure 3.4 shows the architecture of the AgentLint library. The library has two entry points (CLI and MCP server) sharing: a Static Analyzer, a Runtime Analyzer, a Checker Registry (auto-discovers all 14 checkers), a Report Generator (text/JSON/SARIF, where SARIF is the Static Analysis Results Interchange Format used by GitHub and IDEs), and an optional Remediation Adapter for LLM-based fix suggestions. 12 SDK auto-instrumentation packs enable zero-setup runtime analysis. The MCP server was implemented to enable two use cases: IDE integration where developers can query AgentLint results without leaving their editor, and agent self-diagnosis where LLM-based coding assistants that support MCP tool use can invoke AgentLint to detect and suggest fixes for architectural smells in the code they are generating or modifying.

Figure 3.5 illustrates the static analysis pipeline: the CLI loads configuration, the Static Analyzer discovers and parses files, dispatches checkers, and passes results to the Report Generator.

Figure 3.6 shows the runtime analysis flow. The Runtime Interceptor captures each LLM call and tool invocation after completion, dispatches events to runtime checkers, and collects results at session end.

Correctness is verified through 37 testable properties in seven groups (checker invocation, summary accuracy, static detection, runtime detection, configuration

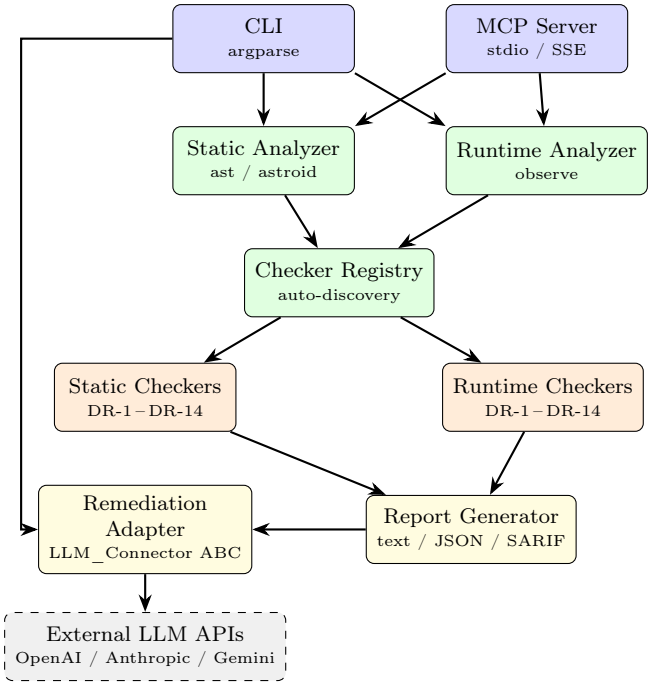


Figure 3.4: C4-level container diagram of the AgentLint library

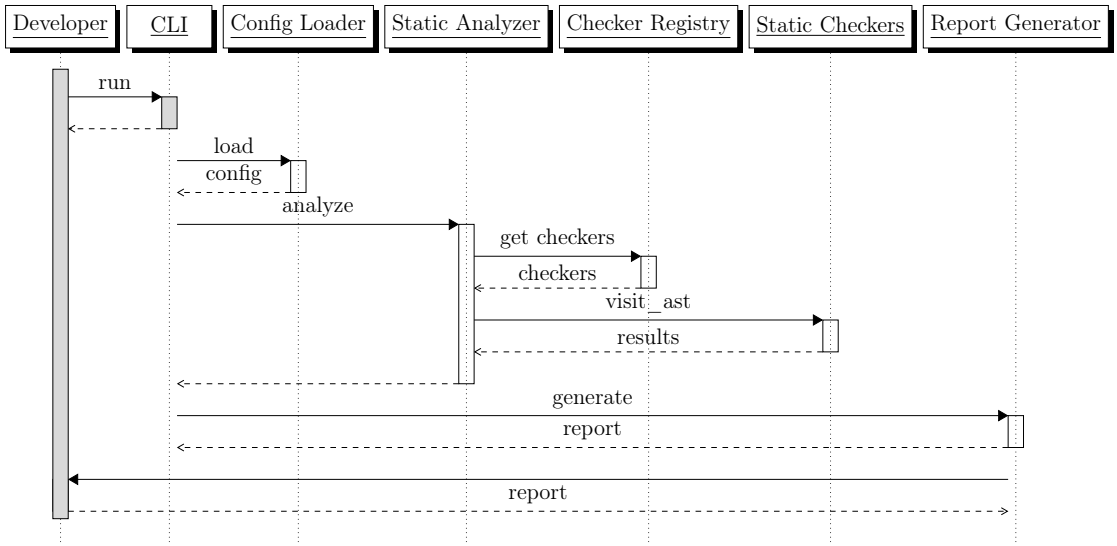


Figure 3.5: Sequence diagram of the static analysis pipeline

handling, report formatting, and model serialization), each tested with 100 random inputs via property-based testing.

Table 3.4 shows the *theoretical* detection capability of AgentLint. The *actual* results in Chapter 4 may cover fewer rules due to scenario limitations.

3.1.4 Phase 4: Demonstration

AgentLint was applied to four frameworks:

1. **PraisonAI:** Multi-agent orchestration built on LiteLLM. Agents are defined

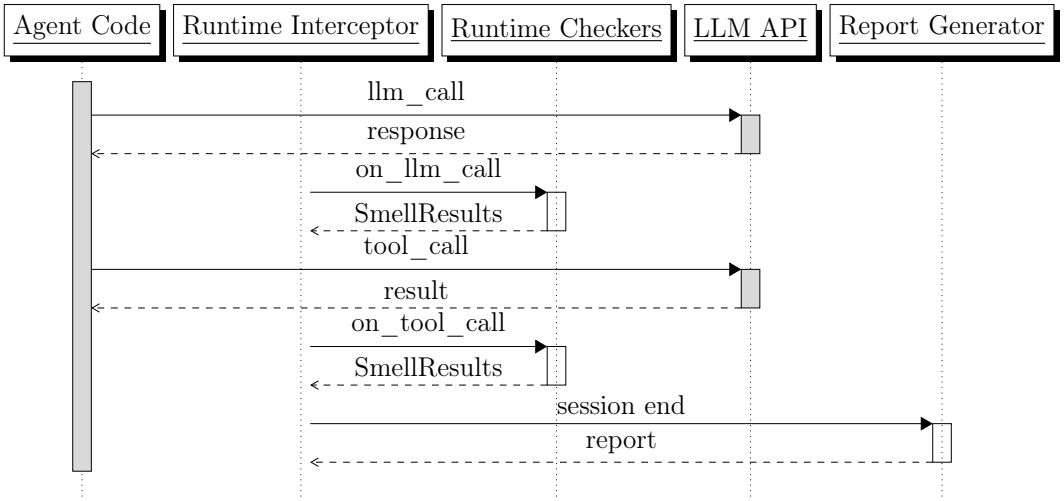


Figure 3.6: Runtime analysis flow: the interceptor captures LLM and tool-call events after completion, routes them to checkers, and generates a report at session end

Table 3.4: Detection coverage of AgentLint across analysis modes

Design Rule	**Static**	**Runtime**
DR-1 Loop Budget	✓	✓
DR-2 Call Dedup	—	✓
DR-3 Context Window	✓	✓
DR-4 Tool Output Cap	✓	✓
DR-5 Persona Pinning	✓	✓
DR-6 Tool Error Contract	✓	—
DR-7 Parallel Isolation	✓	✓
DR-8 Bounded Retry	✓	✓
DR-9 Fallback Path	✓	✓
DR-10 Input Isolation	✓	✓
DR-11 Immutable Templates	✓	✓
DR-12 Synchronized State	✓	✓
DR-13 Serialization Safety	✓	—
DR-14 Deterministic Order	✓	✓
Coverage	**13/14**	**12/14**

declaratively with roles and goals and communicate through a shared task queue. Each agent wraps LLM calls through LiteLLM's unified API, supporting multiple model providers.

2. **PydanticAI:** Type-safe agent framework using Pydantic models for structured I/O. Agents are defined as Python functions with typed parameters; LLM responses are validated against Pydantic schemas before use.

3. **CrewAI:** Crew-based orchestration where agents have roles, goals, and backstories. Tasks are assigned to crews that coordinate through sequential or hierarchical processes; each agent makes independent LLM calls.

4. **DeerFlow:** ByteDance's LangGraph-based research agent. Uses a directed graph of nodes where each node can invoke tools or make LLM calls; supports parallel execution of research sub-tasks.

These four frameworks were selected due to their compatibility with Python, lightweight dependencies, OpenAI SDK compatibility, architectural diversity, and high-relevance smell hits. Figure 3.7 shows the selection pipeline.

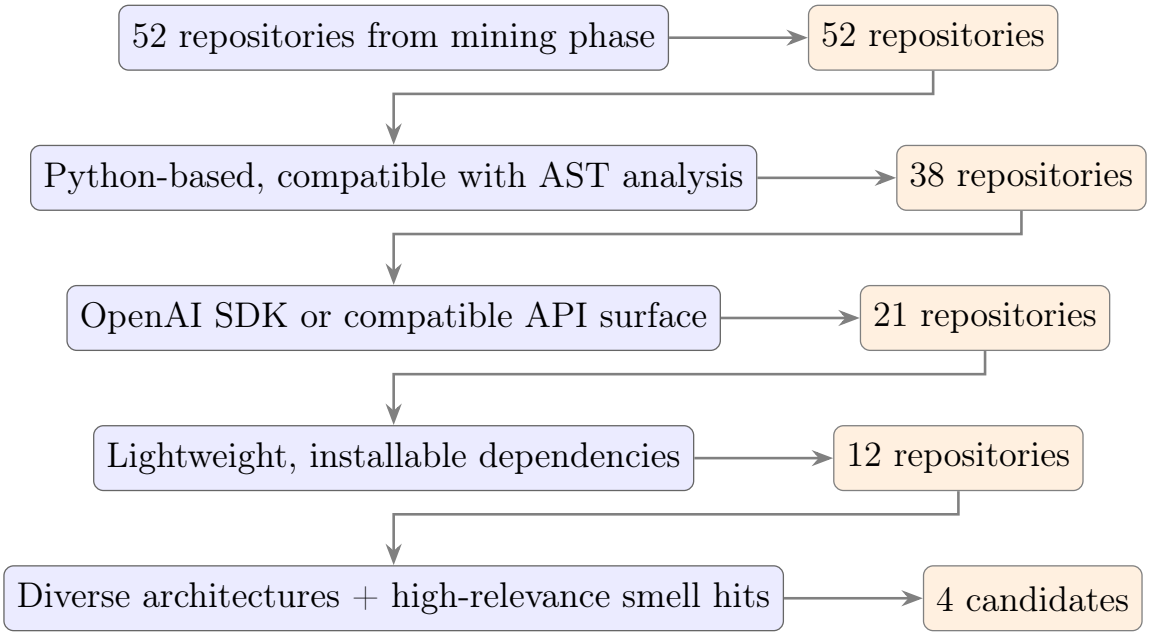


Figure 3.7: Candidate selection pipeline for the evaluation phase

The evaluation was a snapshot analysis at each repository's HEAD revision. Early experiments showed that fixes typically resolved functional bugs while leaving architectural smells in place, making HEAD representative of accumulated technical debt.

3.1.5 Phase 5: Evaluation

The evaluation has two parts: a quantitative detection experiment and a qualitative developer evaluation.

Quantitative Detection Experiment Each repository was analysed with both static and runtime modes. The runtime mode ran a unified test scenario (described in Section 3.1.5) for $n = 50$ trials per repository (200 total). The choice of 50 trials was guided by the Wilson score confidence interval: at this sample size, the interval width is approximately ± 10 percentage points at 95% confidence, which was sufficient since the goal was to determine whether a smell manifests noticeably rather than to estimate its exact frequency. The static mode performed a single scan per repository. Per-trial metrics recorded:

- Smell events captured by the runtime observer
- Event count (LLM calls and tool invocations)

- Smell count and design-rule distribution
- Wall-clock execution time

Static Analysis Static analysis was run with `agentlint -format json` on each repository's source code with a tuned `agentlint.yaml` configuration including repository-specific keyword lists, `model_limit: 8192` tokens (matching gemma3:27b), `max_iterations: 20`, and exclusion patterns for non-agent code. The initial scan produced 374 findings; after tuning exclusions, the count dropped to 279.

Runtime Analysis Runtime analysis used AgentLint's `auto_instrument()` context manager. Each repository was tested with a unified scenario:

- **Phase 1 (Multi-turn conversation):** The agent performs a research task using tool calls. The search tool works for the first three calls, then fails with `ConnectionError`. This grows context toward the 8192-token limit (DR-3).
- **Phase 2 (Model failure):** The agent is reconfigured to use a nonexistent model name and attempts a task, triggering DR-8 (retry without backoff) and DR-9 (no fallback).

The LLM was gemma3:27b on an Ollama GPU server (8192-token context). Each repository was tested for $n = 50$ trials (200 total) with cooldown periods between trials. Figure 3.8 illustrates the two-phase scenario timeline.

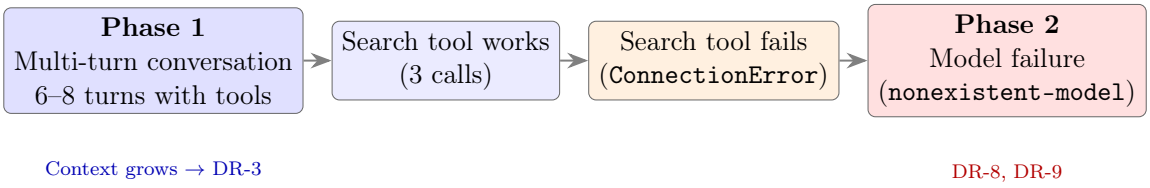


Figure 3.8: Timeline of the unified runtime test scenario. Phase 1 grows context through multi-turn tool use; Phase 2 triggers failure-mode smells.

Qualitative Developer Evaluation The developer evaluation serves Phase 5 of DSRM (Evaluation) and provides practitioner validation that the taxonomy addresses real problems and that the tool output is actionable. While it could support a dedicated research question, it was scoped as a validation activity within the existing research framework to maintain focus on the three RQs.

Eight practitioners at Ericsson's Karlskrona site installed AgentLint, ran it on a sample project, and completed an anonymous 30-item questionnaire covering: background, hands-on verification, detection quality, usefulness, perceived impact , taxonomy completeness and open-ended feedback. The full questionnaire is in Appendix A.

Recruitment for the survey was closed at eight responses as no new themes appeared after the sixth [4]. Likert-scale items (5-point agreement scales from "strongly disagree" to "strongly agree") are analysed with descriptive statistics; open-ended items with thematic coding. The small sample size is a limitation discussed in Section 4.4.

3.1.6 Phase 6: Communication

The results of the study are documented in this thesis and are intended to contribute to both academic research and industrial practice. Additionally, a companion conference paper summarising the taxonomy, tool design, and key evaluation findings has been prepared and is intended for submission to a software engineering venue.

3.2 Validity and Reliability

Validity is supported by combining literature and mining to reduce single-source bias, four architecturally diverse repositories, 50 trials per repository for stochastic variance, and per-project configuration tuning. Limitations include synthetic scenarios, a single LLM model, Python-only scope, and rapid framework evolution. Threats to validity are discussed in Chapter 5.

Chapter 4

Results and Analysis

This chapter presents the results of the AgentLint evaluation in three steps: static detection (Step 1), runtime detection (Step 2), and the comparison between the two modes (Step 3). Section 4.4 reports the developer evaluation results.

4.1 Step 1: Static Detection Capability

AgentLint's static analysis was executed on the four selected frameworks at their current HEAD revisions. The goal was to establish which design rules the static checkers detect and how smell profiles differ across framework architectures.

Per-Repository Static Results Static analysis detected 279 smells spanning 11 of 14 rules across all four repositories. Table 4.1 reports the finding count per repository and design rule.

Table 4.1: Static analysis findings per repository and design rule

Repo	Total	DR-1	DR-2[a]	DR-3	DR-4	DR-5[b]	DR-6	DR-7	DR-8	DR-9	DR-10	DR-11[b]	DR-12	DR-13	DR-14
PraisonAI	153	9	–	19	8	–	62	6	1	22	6	–	4	6	10
PydanticAI	63	–	–	31	2	–	26	–	–	3	–	–	–	–	1
CrewAI	35	–	–	2	–	–	22	–	1	1	2	–	–	–	7
DeerFlow	28	–	–	2	–	–	17	–	–	6	–	–	–	–	3
Total	**279**	**9**	**–**	**54**	**10**	**–**	**127**	**6**	**2**	**32**	**8**	**–**	**4**	**6**	**21**

[a]DR-2 is runtime-only by design. [b]DR-5 and DR-11 were not detected due to checker limitations (see analysis below).

Table 4.2 shows the severity and confidence distribution per repository. Severity indicates potential impact: *error* = likely causes failures in production, *warning* = may cause issues under certain conditions, *info* = best-practice violation with low immediate risk. Confidence indicates detection certainty: *high* = strong structural evidence (e.g., loop without any termination condition), *medium* = heuristic match requiring context (e.g., append call that may have an external guard), *low* = keyword-only match. Practitioners should prioritise high-severity/high-confidence findings and manually review high-severity/low-confidence findings.

Analysis of Static Results The static results show four key patterns:

1. DR-6 (Tool Error Contract) is the most common smell, with 127 of 279 findings (45.5%). It flags tool invocation sites without error handling. Its presence across all four repositories shows that missing tool-call error handling is

Table 4.2: Static findings by severity and confidence per repository

	Severity			Confidence		
Repo	**Error**	**Warning**	**Info**	**High**	**Medium**	**Low**
PraisonAI	77	60	16	86	45	22
PydanticAI	26	36	1	24	32	7
CrewAI	24	4	7	23	10	2
DeerFlow	17	8	3	11	15	2
Total	**144**	**108**	**27**	**144**	**102**	**33**

widespread in agent frameworks, consistent with the tool-description quality issues reported by Hasan et al. [7].

2. DR-3 (Context Bloat) is second with 54 findings (19.4%). PydanticAI accounts for most of these (31) because it implements multiple model backends (Anthropic, Bedrock, Gemini, OpenAI), each with its own message-history logic containing unbounded `append()` calls. This aligns with the theoretical prediction by Yadav et al. [20] that unbounded context accumulation is a systemic risk in agent architectures.
3. PraisonAI triggers all 11 detectable rules, showing the broadest smell profile. DeerFlow triggers only 4 rules with 28 total findings, suggesting a cleaner design. The raw counts are not normalised by repository size because the analysis targets agent-specific modules only (non-agent code is excluded via configuration). For context, PraisonAI's analysed surface (∼12K LoC in agent modules) is approximately 3× larger than DeerFlow's (∼4K LoC), while its finding count is 5.5× higher, suggesting genuinely higher smell density rather than merely a larger codebase.
4. Three rules were not detected: DR-2 (Call Deduplication) is runtime-only by design; DR-5 (Persona Pinning) was missed because the checker looks for `system=` keyword arguments but these frameworks use message lists; DR-11 (Immutable Templates) was missed because the checker does not recurse into nested blocks. DR-5 and DR-11 are checker limitations, not absence of the smell. These limitations were discovered during the evaluation when the checkers produced zero findings despite manual inspection confirming the smells exist in forms the checkers do not recognise. It was not feasible to redesign the checkers at that stage, so they are acknowledged as limitations. DR-2's runtime-only nature was known from the design phase.

To illustrate what detected smells look like in practice, the following synthetic examples show representative patterns from the three most common rules:

Listing 4.1: DR-6: Tool call without error handling

```
# Detected: tool invocation with no try/except
result = search_tool.invoke(query)
agent.memory.append(result)  # fails if invoke raises
```

Listing 4.2: DR-3: Unbounded message history growth

```
while not task_complete:
    response = llm.chat(messages)
    messages.append(response)  # no size guard
    tool_output = execute_tool(response)
    messages.append(tool_output)  # grows without bound
```

Listing 4.3: DR-9: Missing fallback path

```
def run_agent(task):
    response = llm.generate(task)
    return response  # no fallback if model fails/returns empty
```

4.2 Step 2: Runtime Detection Capability

The second step runs AgentLint's runtime analysis using the test scenario described in Section 3.1.5. Each repository was tested for 50 trials (200 total).

Per-Repository Runtime Results Table 4.3 shows the runtime detection results.

Table 4.3: Runtime analysis results per repository (50 trials each)

Repo	Avg Events	Total Smells	DR-1	DR-3	DR-8	DR-9
PraisonAI	9.2	98	–	54	–	44
PydanticAI	7.0	58	–	58	–	–
CrewAI	5.2	57	–	11	–	46
DeerFlow	13.3	55	5	–	50	–
Total	—	**268**	**5**	**123**	**50**	**90**

Runtime analysis detected 268 smell instances across 200 trials, covering 4 of 14 design rules: DR-1 (Loop Budget), DR-3 (Context Bloat), DR-8 (Bounded Retry), and DR-9 (Fallback Path). Figure 4.1 shows how smell counts vary across trials per repository.

Analysis of Runtime Results The runtime results show framework-specific patterns:

DR-3 (Context Bloat) was detected in PraisonAI, PydanticAI, and CrewAI but not DeerFlow. PydanticAI triggered it most consistently (58 instances across 50 trials). The checker fires when token usage exceeds 85% of the 8192-token limit, and PydanticAI consistently approached this threshold during multi-turn conversations. This confirms the theoretical prediction by Yadav et al. [20] that unbounded context accumulation degrades reasoning quality proportionally to context length. DeerFlow avoided DR-3 because LangGraph's graph-based state model handles context more efficiently, consistent with the modular design principles described by Wang et al. [17].

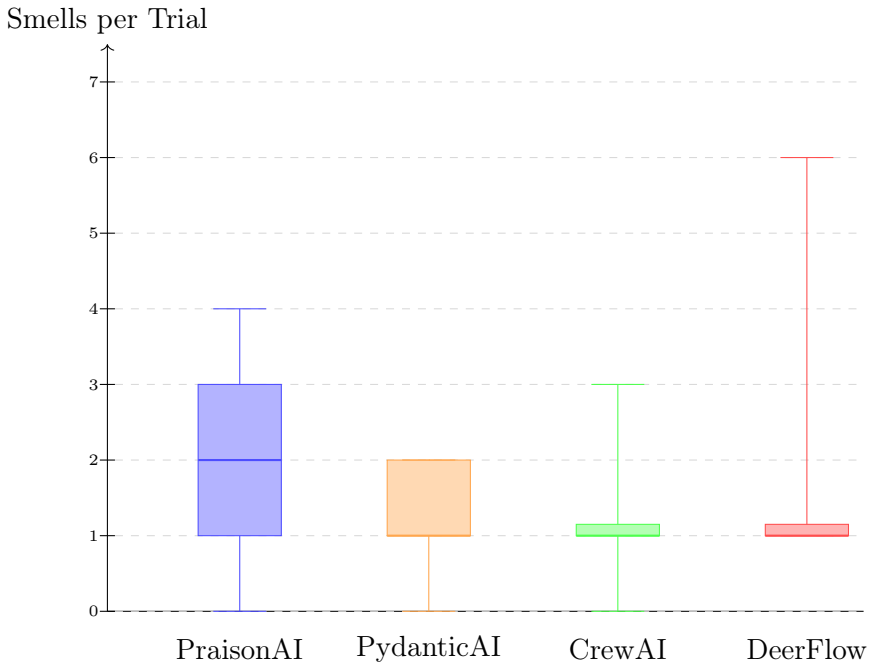


Figure 4.1: Distribution of smell counts per trial (50 trials per repository)

DR-9 (Fallback Path) was detected in PraisonAI (44/50 trials) and CrewAI (46/50 trials). PraisonAI's inconsistent rate is explained by LiteLLM's error handling: in some trials, LiteLLM catches the model-not-found error internally before AgentLint's interceptor can observe it. This pattern mirrors the error-swallowing behaviour documented by Islam et al. [8].

DR-8 (Bounded Retry) was detected only in DeerFlow, in all 50 trials. LangGraph's retry mechanism immediately retries failed calls without backoff, and the checker detected the 0.00-second interval against the expected 2.00-second threshold.

DR-1 (Loop Budget) was detected only in DeerFlow, in 5 of 50 trials (10%). These trials had unusually high event counts (25 events vs. the typical 13), where the agent exceeded the configured `max_iterations: 20` limit. The low rate reflects LLM non-determinism: Suri et al. [15] showed that reasoning instability correlates with non-converging trajectories, and the 10% manifestation rate confirms that loop-budget violations are stochastic rather than deterministic.

Inferential Analysis Wilson score intervals at 95% confidence confirm detection rates are genuine: PraisonAI DR-9 at 88% [75.7%, 95.5%], CrewAI DR-9 at 92% [80.8%, 97.8%], DeerFlow DR-8 at 100% (lower bound 92.9%), DeerFlow DR-1 at 10% [3.3%, 21.8%]. Fisher's exact test shows no significant difference between PraisonAI and CrewAI DR-9 rates ($p = 0.74$), suggesting that the underlying architectural vulnerability is comparable across crew-based orchestration patterns [8].

Spearman rank correlation between event count and smell count yields $\rho = 0.31$ ($p < 0.001$), indicating that framework architecture matters more than event volume. This aligns with the finding by Hajdu et al. [5] that structural design choices dominate over workload intensity in determining defect manifestation rates.

4.3 Step 3: Static vs. Runtime Comparison

The third evaluation step compares the detection coverage of static and runtime analysis to answer RQ3. Table 4.4 presents a side-by-side comparison of which design rules each mode detected.

Table 4.4: Design rule detection coverage: static vs. runtime analysis

DR	Name	S	R	Cov.	DR	Name	S	R	Cov.
DR-1	Loop Budget	✓	✓	Both	DR-8	Bounded Retry	✓	✓	Both
DR-2	Call Dedup	—	—	—[a]	DR-9	Fallback Path	✓	✓	Both
DR-3	Context Window	✓	✓	Both	DR-10	Input Isolation	✓	—	S only
DR-4	Tool Output Cap	✓	—	S only	DR-11	Immutable Templ.	—	—	—[b]
DR-5	Persona Pinning	—	—	—[b]	DR-12	Sync. State	✓	—	S only
DR-6	Tool Error	✓	—	S only	DR-13	Serial. Safety	✓	—	S only
DR-7	Parallel Isol.	✓	—	S only	DR-14	Determ. Order	✓	—	S only

Total: Static 11/14, Runtime 4/14, Combined 11/14

[a]Runtime-only by design; scenario did not trigger. [b]Checker limitation (see Discussion).

Analysis: Complementary Detection Figure 4.2 summarises the overlap between the two modes.

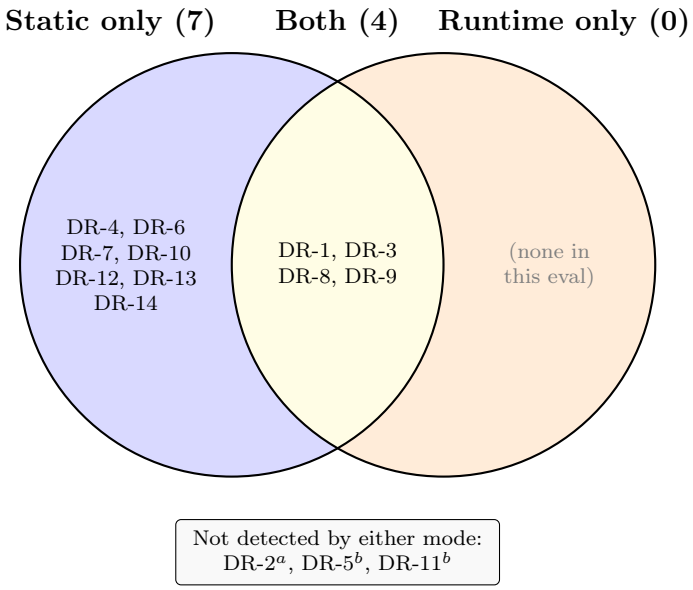


Figure 4.2: Venn diagram of design rule detection coverage. Static detects 11 rules, runtime 4, with 4 in overlap.

For DR-3, static finds unbounded `messages.append()` patterns while runtime confirms actual context overflow. For DR-9, static finds single-provider call sites while runtime confirms failure without fallback. Seven rules were detected only statically as their patterns did not produce observable runtime events. No rule was detected only by runtime. Runtime analysis did not detect rules beyond those found by static analysis. Its value lies in providing a different type of information: while static analysis identifies that a structural pattern exists, runtime analysis provides

evidence of manifestation and quantifies how frequently the pattern causes actual failures. For example, knowing that an error-handling smell exists in code is one type of information; knowing that it manifests in 88% of executions is qualitatively different and more actionable for prioritisation. This distinction is particularly important in agent systems where non-deterministic LLM behaviour means not every structural pattern manifests in every execution. This complementarity supports the dual-mode rationale: Hajdu et al. [5] reported that combining static and dynamic analysis improves defect detection by 76%, and the present results confirm that the two modes address fundamentally different aspects of agent quality.

4.4 Developer Evaluation Survey

This section reports the results of the developer evaluation described in Section 3.1.5. Eight practitioners (P1–P8) at Ericsson's Karlskrona site completed the questionnaire between April and May, 2026. The saturation justification and sample-size limitation are discussed in Section 3.1.5 and Section 4.4, respectively.

Participant Demographics Tables 4.5 and 4.6 summarise participant backgrounds. Most participants (62.5%) have 3–10 years of experience. The sample is moderately familiar with both agent systems (mean 3.13/5) and linting tools (mean 3.63/5).

Table 4.5: Participant background ($n = 8$)

Category	**Count**
Years of experience	
<1 year	1
1–3 years	1
3–5 years	1
5–10 years	4
>10 years	1
Role	
Software Engineer	4
AI/ML Engineer	2
Software Architect	2

Table 4.6: Participant familiarity and tool usage ($n = 8$)

Measure	**Mean**	**Median**	**Range**
Agent system familiarity (Q2)	3.13	3	2–4
Linting tool familiarity (Q3)	3.63	3.5	2–5
Ran on own code (Q6)	Yes: 3; No: 5		

Hands-on Task Verification All eight participants confirmed running AgentLint (non-empty smell counts in Q5). Three also ran it on their own code.

Detection Quality Participants rated detection quality positively. Location accuracy scored highest, followed by report clarity. Correspondence with expected problems was strong and with real agent problems slightly lower. Severity calibration received the lowest score, suggesting room for improvement. Most participants did not find the tool too noisy, though opinions varied.

A total of 37.5% of participants saw no false positives, whereas 25% saw 1 or 2, another 25% saw 3 to 5, and 12.5% saw more than five false positives. DR-14 (Deterministic Ordering) was the most cited source of false positives (37.5%), with P7 noting it "flagged every set iteration even when ordering didn't matter." DR-8 (Bounded Retry) was cited by 25% of participants.

One participant (P6) reported missed issues (Q15), identifying DR-7, DR-10, and DR-12 as incomplete on the sample project. P7 noted that AgentLint does not cover prompt quality, token efficiency, or hallucination prevention, but framed these as out-of-scope rather than missed detections.

Usefulness and Usability Table 4.7 shows the usability Likert results. Participants agreed the tool provides a way to find architectural issues (Q17, 4.00) and that the CLI output is clear (Q19, 4.00). Adoption intent (Q18, 3.75) is positive. Installation (Q20, 3.63) had the widest disagreement; P7 noted it "needs Python 3.10+ but that's not obvious until it fails."

Table 4.7: Likert-scale results for usefulness and usability (Q17–Q20), $n = 8$, scale 1–5

#	Item	Mean	Mdn	SD
Q17	Systematic way to identify architectural issues	4.00	4.0	0.53
Q18	Would use this tool in my workflow	3.75	4.0	0.71
Q19	CLI output was clear and well-organised	4.00	4.0	0.76
Q20	Installation was straightforward	3.63	4.0	1.19

Tables 4.8 and 4.9 show which rules participants found most and least useful. DR-3 (Context Window) and DR-10 (Input Isolation) tied for most useful (5 selections each). DR-14 (Deterministic Ordering) and DR-13 (Serialization Safety) tied for least useful (3 each). DR-6 (Tool Error Contract) appeared in both columns, reflecting divided opinion.

Being ranked as "noisy" does not imply a rule is not useful. DR-6 (Tool Error Contract) appears in both the most-useful and noisiest lists, reflecting that participants value the detection but find the volume of findings overwhelming in large codebases. Noisiness indicates a need for better filtering or configuration rather than removal of the rule.

Perceived Impact Participants rated confidence that fixing detected smells would improve reliability at 3.75/5 (Q23) and reduce token costs at 3.38/5 (Q24). Most participants (62.5%) rated reliability confidence higher than cost confidence. Figure 4.3 shows which smell categories participants have experienced in production (Q25). Memory/context bloat was the most common (75%), followed by error recovery gaps (62.5%). No participant reported state management issues in production.

Table 4.8: Design rules ranked by usefulness ($n = 8$)

Rule	Votes
DR-3 Context Window	5
DR-10 Input Isolation	5
DR-1 Loop Budget	4
DR-6 Tool Error	3
DR-2 Call Dedup	1
DR-7 Parallel Isol.	1
DR-9 Fallback Path	1
DR-14 Determ. Order	1

Table 4.9: Design rules ranked by noisiness ($n = 8$)

Rule	Votes
DR-13 Serial. Safety	3
DR-14 Determ. Order	3
DR-2 Call Dedup	2
DR-4 Tool Output Cap	2
DR-6 Tool Error	2
DR-11 Immutable Templ.	2
DR-1 Loop Budget	1
DR-7 Parallel Isol.	1
DR-8 Bounded Retry	1
DR-10 Input Isolation	1

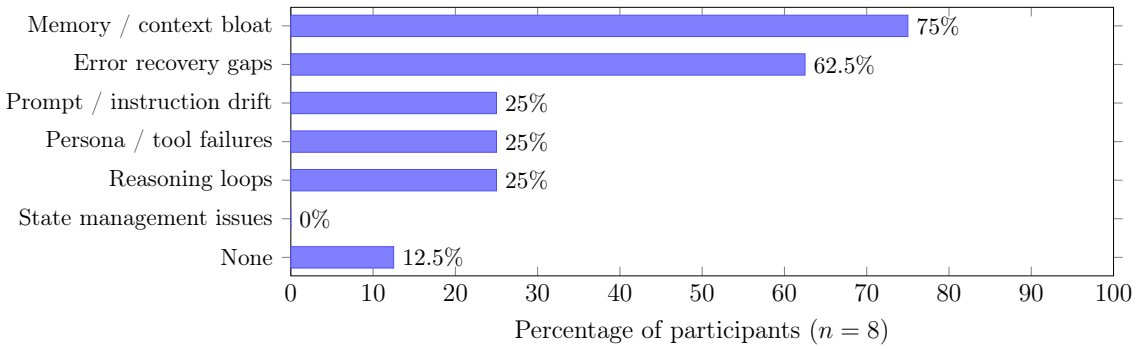


Figure 4.3: Q25: smell categories participants have experienced in production (multi-select, $n = 8$).

Taxonomy Completeness Five of eight participants (62.5%) said the taxonomy does not cover all problems they have encountered. Four described specific gaps:

- **Output validation** (P4): no rule checks whether the agent validates LLM responses before acting on them.
- **Per-call timeouts** (P4): DR-8 covers retry backoff and DR-1 covers loop bounds, but neither checks for missing timeouts on individual API calls.
- **Secret/Personally Identifiable Information (PII) leakage** (P6, P8): no rule detects credentials or personal data being embedded in prompts or logs.

- **Hallucination in structured output** (P7): the taxonomy does not cover agents fabricating non-existent file paths, endpoints, or command flags.
- **Budget/cost ceilings** (P8): no rule enforces a maximum token or cost budget per request.

These five items are treated as candidate future design rules in Section 6.

Open-Ended Feedback (Q28–Q30) **Most valuable (Q28):** detection of agent-specific issues that generic linters miss, accurate locations, and fast feedback. **Most missing (Q29):** actionable fix guidance (`-fix` mode), easier configuration/suppression, and prompt-level analysis. **Other (Q30):** P4 called it "impressive for a v0.1.0 release"; P7 noted DR-14 noise and undocumented Python 3.10+ requirement.

Limitations of the Developer Evaluation The sample ($n = 8$) is below the 10–15 target and limited to one industrial site. The hands-on task used a small sample project, so issues at scale are not captured.

Chapter 5

Discussion

This chapter interprets the results, relates them to existing literature, and discusses limitations.

5.1 Answering the Research Questions

RQ1: Taxonomy of Architectural Smells The taxonomy comprises six categories with 14 design rules derived from two complementary sources: a literature review of LLM agent failure patterns and repository mining across 52 open-source projects. This approach strengthens the taxonomy's grounding: categories such as Reasoning Loops and Memory/Context Bloating emerged independently from both literature [1, 20] and mining evidence, while Prompt/Instruction Drift was primarily literature-driven [3] and State Management Issues primarily mining-driven.

The evaluation confirms broad coverage: static analysis detected 11 of 14 rules, and runtime detected 4. Three rules (DR-2, DR-5, DR-11) were not detected by either mode, which reflects scenario limitations rather than taxonomy flaws. These rules target patterns (duplicate tool calls, persona drift, mutable templates) that the test scenarios did not trigger.

The developer survey adds practitioner validation: 62.5% identified at least one gap (output validation, per-call timeouts, secret/PII leakage, hallucination in structured output, cost ceilings), but these extend the taxonomy rather than contradict it. Notably, the suggested extensions map to concerns that were either out-of-scope in the mining phase (cost management, PII) or emerged after the mining period (structured-output hallucination). This suggests the taxonomy captures the core architectural smells present at the time of construction, while remaining extensible as the domain evolves. By "core" we mean patterns that are (a) specific to LLM-based agent architectures and not covered by existing linters, (b) empirically observed in multiple independent repositories or documented in peer-reviewed literature, and (c) detectable through code analysis. The five participant-identified gaps extend this core with concerns that were out-of-scope or emerged after the mining period.

The most frequently experienced smell category in production was Memory/-Context Bloat (75% of participants), followed by Error Recovery Gaps (62.5%). No participant reported State Management Issues in production, which may reflect either the relative rarity of multi-agent deployment experience among participants or the difficulty of attributing state-related failures to architectural causes.

RQ2: Detection of Architectural Smells AgentLint demonstrates that architectural smells in LLM-based agent systems are detectable through automated analysis. Static analysis provides broad coverage (11/14 rules) with precise file and line locations, enabling developers to identify problematic patterns during code review. Runtime analysis provides behavioural confirmation for fewer rules (4/14) but captures phenomena that only manifest under LLM non-determinism. For example, PraisonAI's DR-9 (No Fallback) manifested in 88% of trials, meaning 12% of runs would appear healthy despite the architectural vulnerability existing in code.

Practitioner feedback supports detection quality: report clarity scored 4.25/5 and location accuracy 4.50/5, indicating that developers can act on the findings without extensive manual investigation. The main precision weakness is DR-14 (Deterministic Ordering), cited as a false-positive source by 37.5% of participants. This suggests that rules targeting non-determinism require more context-aware heuristics to distinguish intentional from accidental ordering dependencies.

The dominance of DR-6 (Tool Error Contract) at 45.5% of all static findings reveals a systemic gap: agent frameworks consistently lack error handling around tool invocations. This is not a detection artefact but reflects a genuine pattern where frameworks prioritise the happy path and defer error handling indefinitely.

RQ3: Runtime vs. Static Analysis Effectiveness The combined approach covers 11/14 rules. Runtime adds behavioural confirmation that static cannot provide: it shows whether a code-level pattern actually causes problems during execution. Runtime's lower coverage (4/14) has two causes: seven rules target code-level patterns that do not produce observable events (e.g., missing error handling is a code absence, not a runtime event), and three rules need conditions the test scenario does not trigger.

The two modes overlap on four rules (DR-1, DR-3, DR-8, DR-9). For these, static identifies the structural pattern and runtime confirms the behavioural manifestation. This overlap is valuable rather than redundant because it provides confidence that the detected patterns are not theoretical but actually manifest in practice. PraisonAI's 88% and CrewAI's 92% DR-9 manifestation rates demonstrate that code-level smells translate to observable failures at high rates under realistic conditions.

Runtime analysis is complementary to static analysis rather than a replacement. Static is fast, deterministic, and suitable for CI pipelines. Runtime requires LLM inference, is stochastic, and better suited for periodic integration testing. The practical recommendation is to use static analysis as a continuous gate and runtime analysis as a periodic validation step.

5.2 Framework-Specific Observations

- **PraisonAI** has the most findings (153 static, 11/11 rules). Its DR-9 rate (44/50) is lower than CrewAI's (46/50) because LiteLLM sometimes swallows provider errors before AgentLint can observe them.
- **PydanticAI** is dominated by DR-3 (31 findings, 49%) and DR-6 (26, 41%). Its fail-fast design raises an exception immediately on model failure, explaining

the absence of DR-9 findings.

- **CrewAI** has few static findings (35) but high runtime DR-9 detection (46/50). Its error handling is concentrated in one module (`tool_usage.py`), making remediation tractable.

- **DeerFlow** is the cleanest framework (28 static findings). LangGraph's graph-based state management avoids DR-3 but introduces DR-8 (retry without back-off) in every trial. This illustrates that architectural decisions solving one smell category can introduce another.

These profiles reveal a tension in agent framework design: no single architecture eliminates all smell categories. PraisonAI's flexibility comes at the cost of a broad smell surface. DeerFlow's graph-based design solves context management but introduces retry issues. PydanticAI's strict typing prevents certain errors but makes others invisible to runtime observation. Smell detection tools must therefore be framework-aware, and remediation strategies should match each framework's design philosophy.

The concentration of findings also varies significantly. CrewAI's 20/22 DR-6 findings in a single module (`tool_usage.py`) means that one targeted refactoring addresses 91% of its tool-error issues. In contrast, PraisonAI's DR-6 findings are distributed across 12 modules, requiring a broader remediation effort. This difference has practical implications for how development teams should prioritise fixes: concentrated smells are cheaper to fix but may indicate a single point of failure, while distributed smells indicate systemic design patterns that require architectural-level changes.

5.3 Unexpected Findings

The evaluation produced three results that were not anticipated during the design phase.

First, DeerFlow produced zero DR-3 (Context Bloat) findings despite having the highest event count per trial (13.3 events on average). LangGraph uses a graph-based state model where each node maintains its own bounded context, rather than accumulating messages in a shared history. Even with many events, no single context window grows unboundedly. Architectural decisions at the framework level can thus eliminate entire categories of smells.

Second, PydanticAI produced no DR-9 (Fallback Path) findings despite lacking any fallback mechanism. The reason is its fail-fast design: when a model call fails, PydanticAI immediately raises a Python exception that terminates the agent. The AgentLint runtime interceptor observes events *after* they complete, so the exception propagates before the interceptor can record the absence of a fallback. The smell exists in the architecture but is invisible to the current detection approach. Post-hoc runtime observation cannot detect smells defined by what *did not* happen, which would require a different instrumentation strategy.

Third, DeerFlow's DR-1 (Loop Budget) rate was only 10% (5 of 50 trials). The configured limit was 20 iterations, and the LLM typically terminated its reasoning

within budget. The five violations occurred in trials where the LLM generated unusually long reasoning chains (25+ events). Smell manifestation in agent systems thus depends on three factors: scenario design, framework architecture, and LLM non-determinism.

5.4 Configuration Sensitivity

The runtime `model_limit` parameter was set to 8192 tokens to match the context window of gemma3:27b (the LLM used in all trials). This calibration is essential: without it, the DR-3 checker would never fire because the default threshold assumes a larger context window. Similarly, for static analysis, the initial scan produced 374 findings, but after configuring exclusion patterns to filter non-agent code paths (test files, documentation generators, build scripts), the count dropped to 279 (a 25.4% reduction). These results show that smell detection must be calibrated to the specific deployment environment, including model parameters, framework conventions, and project structure.

5.5 Checker Limitations

Two static checkers failed to detect their target smells due to implementation gaps.

The DR-5 (Persona Pinning) checker looks for LLM calls where the `system=` keyword argument is missing after a tool call. All four frameworks use message lists instead of keyword arguments, so the pattern does not match. The smell is likely present but the checker cannot see it in this form.

The DR-11 (Immutable Templates) checker detects reassignment of prompt template strings but does not recurse into nested `if`/`for` blocks. Template mutations inside control flow are missed. Fixing this would require deeper AST traversal with scope tracking.

Precision and Recall Assessment A formal precision/recall evaluation would require a manually annotated ground-truth dataset, which was not feasible in this thesis. The available evidence supports a qualitative assessment.

Precision. In Q13, 37.5% of participants saw zero false positives, 25% saw one to two, and 12.5% saw more than five. DR-14 and DR-13 were the main sources of false positives. The 25.4% reduction after configuration tuning (374 to 279 findings) shows that per-project configuration is needed for acceptable precision.

Recall. One participant (P6) identified missed detections for DR-7, DR-10, and DR-12. Three rules (DR-2, DR-5, DR-11) were not detected by either mode due to scenario limitations or checker gaps. Combined recall is bounded at 11/14 (78.6%) for these repositories, with actual recall likely lower due to within-rule false negatives.

A formal study with annotated ground truth is identified as future work. The evidence suggests precision is stronger than recall in the current implementation.

5.6 Implications for Practice

Four practical implications follow from the evaluation.

First, DR-6 (Tool Error Contract) accounts for 45.5% of all static findings across all four repositories. Missing error handling around tool invocations was the most prevalent pattern in the evaluation and was also observed in 28 of 36 repositories with smell hits during the mining phase. While this suggests the pattern is widespread in the Python agent ecosystem, the claim is limited to the frameworks examined in this study.

Second, static and runtime analysis fit different parts of the development workflow. Static analysis works as a fast gate in pull-request checks, providing immediate feedback with zero runtime cost. Runtime analysis suits periodic integration tests where LLM behaviour can be observed over multiple runs.

Third, smell detection tools need per-project calibration. The 25.4% reduction in findings after configuration tuning (374 to 279) shows that uncalibrated tools produce too much noise for practical use.

Fourth, smell reports should be interpreted relative to a framework's design philosophy. CrewAI's 20/22 DR-6 findings sit in one module, so a single refactoring addresses 91% of its tool-error issues. PraisonAI's findings are spread across 12 modules, requiring broader changes.

5.7 Ethical and Societal Implications

Environmental cost. The runtime evaluation ran 200 trials with a 27B-parameter model (gemma3:27b). The model was accessed via a shared Ollama GPU server, so inference cost is shared across users. Still, researchers running similar evaluations should consider the cumulative energy cost and report compute budgets.

Responsibility and liability. If AgentLint flags an issue that a developer ignores and the system later fails, questions of responsibility arise. Conversely, false negatives in safety-critical contexts (medical, financial) could have consequences beyond software quality. The tool's documented gaps should inform risk assessments.

Accessibility. AgentLint is Python-based, which lowers adoption barriers. However, the configuration burden (per-project YAML tuning, undocumented Python 3.10+ requirement) may exclude less-resourced teams.

Positive impact. Improving agent reliability and reducing unnecessary token consumption contributes to more sustainable AI deployment.

5.8 Relation to Existing Literature

[5] reported a 76% increase in defect detection when combining static and dynamic analysis. The present evaluation does not achieve a comparable improvement in rule count (11/14 combined, same as static alone), but runtime adds behavioural confirmation that code patterns actually cause problems during execution. This distinction matters in the LLM domain, where non-deterministic behaviour means not every pattern manifests in every run. The value of runtime analysis in this context is not additional coverage but additional confidence: knowing that a pattern

manifests in 88% of runs (PraisonAI DR-9) is qualitatively different from knowing only that the pattern exists in code.

The smell distribution aligns with the mining phase. Error Recovery/Resilience was the most prevalent mining category (368 hits across 52 repositories), and DR-6 dominates the static findings. Memory/Context Bloating was second in mining (221 hits), and DR-3 is second in static findings (54 instances). This consistency between the mining phase (which analysed historical commits) and the detection phase (which analysed current code) strengthens the taxonomy's construct validity: the same patterns that developers have historically struggled with remain present in current codebases.

The framework-specific results show that architectural choices involve trade-offs between smell categories. This extends [12]'s modular perspective by showing that module implementation choices have measurable consequences for smell manifestation. Where Shang et al. describe what modules agents *should* have, this thesis shows what goes wrong when those modules are implemented without quality safeguards.

The combined-analysis approach connects to the broader static analysis literature. Traditional tools like FindBugs and PMD detect code-level patterns but cannot confirm whether those patterns cause runtime failures [22]. AgentLint's runtime mode addresses this gap by observing whether structural patterns produce behavioural symptoms during execution. This is particularly important for agent systems because the LLM layer introduces a non-deterministic boundary between code structure and runtime behaviour that does not exist in traditional software.

The developer evaluation results (report clarity 4.25/5, location accuracy 4.50/5) compare favourably with Britto et al.'s MLpylint evaluation [6], which reported similar usability scores for ML-specific linting. This suggests that the Design Science approach of iterating tool design based on practitioner feedback produces tools that developers find actionable, regardless of the specific domain (ML pipelines vs. agent architectures).

5.9 Limitations and Threats to Validity

Internal validity. The test scenario is synthetic and does not represent production workloads. It was designed to trigger specific rules (DR-3 through context growth, DR-8 and DR-9 through model failure), so the runtime results partly reflect scenario design rather than natural smell occurrence. The 4/14 runtime detection rate is a lower bound on what runtime analysis can detect. Using gemma3:27b as the only model introduces bias. Its 8192-token context window is smaller than production models (GPT-4's 128K, Claude's 200K), which may inflate DR-3 detection rates.

External validity. The evaluation covers four Python-based agent frameworks, which limits generalisability to other languages and frameworks. The 50-trial design captures stochastic variance but may not be sufficient for rare events. DR-1 was detected in only 5 of 50 DeerFlow trials (10%), suggesting that some smells manifest infrequently.

Construct validity. The `model_limit` configuration directly affects DR-3 detection rates. The choice of 8192 tokens means that context bloat is detected earlier than it would be with a larger model. The DR-8 checker was refined during the

evaluation; final results reflect the corrected version. The four evaluation frameworks were among the 52 mined repositories. However, the mining phase analysed historical commit data (past code changes) while the evaluation ran on the current HEAD of each repository (present code state). The tool detects smells in the current codebase through AST patterns, not in the historical commits that informed the taxonomy. This temporal separation mitigates the independence concern.

Researcher bias. Both authors are software developers at Ericsson, which may introduce bias toward code-level issues over organisational or process-level concerns. The developer evaluation partially mitigates this by incorporating external practitioner perspectives.

Temporal validity. The agent framework ecosystem evolves rapidly. The evaluated repositories reflect their state as of April 2026; subsequent refactoring or version updates may have addressed some detected smells or introduced new ones. Results should be interpreted as a snapshot rather than a permanent characterisation.

Configuration sensitivity. AgentLint's detection results depend on the `agentlint.yaml` configuration, including keyword lists, exclusion patterns, and threshold values. Different configurations may yield different finding counts. The per-project tuning described in Section 3.1.5 improves precision but limits reproducibility without the exact configuration files.

Chapter 6

Conclusions and Future Work

This thesis contributes a taxonomy of 14 architectural design rules in six categories constructed through literature review and repository mining across 52 frameworks, AgentLint as a Python library implementing the taxonomy through combined static and runtime analysis, and an empirical evaluation on four frameworks with 200 runtime trials and a developer evaluation with eight practitioners.

Answering the Research Questions **RQ1 (taxonomy).** The 14-rule taxonomy in six categories is empirically grounded and broadly validated. Static analysis confirmed 11/14 rules detectable; practitioners identified five candidate extensions that complement rather than contradict the existing rules.

RQ2 (detection). AgentLint detects architectural smells in real frameworks. Static analysis produced 279 findings (DR-6 dominant at 45.5%), runtime confirmed manifestation across 200 trials, and practitioners rated report clarity at 4.25/5 and location accuracy at 4.50/5.

RQ3 (runtime versus static). The two modes are complementary. Static provides broad structural coverage (11/14 rules); runtime adds behavioural confirmation under LLM non-determinism (4/14 rules, with 88–92% manifestation rates for DR-9). Neither mode alone achieves what the combination provides.

Limitations The conclusions are qualified by a single LLM model on a single scenario across four Python frameworks, $n = 8$ participants at one site, documented checker gaps bounding recall at 11/14, and three rules unexercised at runtime.

Future Work The evaluation and the developer feedback converge on five directions for future work.

First, the taxonomy should be extended with the candidate rules identified in the developer evaluation (Section 6). Output validation, per-call timeouts, credential and PII handling in prompts, hallucination in structured output, and per-request cost budgets together form a roadmap for a future taxonomy revision.

Second, the checker precision weaknesses identified in the survey should be addressed. The DR-14 checker should be extended with ordering-sensitivity analysis; the DR-11 checker should recurse into nested `if`/`for` blocks; and the DR-5 checker should recognise message-list-based LLM calls.

Third, actionable remediation support was the most consistent request. A `-suggest` or `-fix` mode would address the gap that 50% of participants identified.

Fourth, ergonomic improvements: inline suppression comments, default exclusion of virtual environments, deduplication of findings, and clearer Python 3.10+ documentation.

Fifth, the evaluation should be broadened with extended runtime scenarios, a multi-model study, and a longitudinal study tracking smell counts before and after remediation.

Candidate Design Rules from the Developer Evaluation Five candidate rules emerged from the survey:

- **DR-15 (LLM Output Validation).** Validate LLM responses before downstream use.
- **DR-16 (Per-Call Timeouts).** Require explicit timeouts on every LLM and tool call.
- **DR-17 (Secret/PII Containment).** Detect credentials or PII in prompts, logs, or memory.
- **DR-18 (Structured-Output Hallucination).** Detect fabricated file paths, endpoints, or flags.
- **DR-19 (Per-Request Budget Ceiling).** Enforce maximum token or cost budget per request.

Adding these would bring the taxonomy to 19 rules across seven categories.

Closing Remarks This thesis has shown that architectural smells in LLM-based agent systems can be classified into a taxonomy and detected through combined static and runtime analysis. The taxonomy provides a shared vocabulary, the tool provides automated evidence, and the evaluation provides initial validation. Together, they move the conversation about agent reliability from anecdotal failure reports toward systematic quality assessment.

References

[1] T.-H. P. Chen *et al.*, "Towards structured, state-aware, and execution-grounded reasoning for software engineering agents," 2026. [Online]. Available: https://arxiv.org/abs/2602.04640

[2] R. Ehsani *et al.*, "Where do AI coding agents fail? an empirical study of failed agentic pull requests in GitHub," in *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR)*. Rio de Janeiro, Brazil: ACM, 2026. [Online]. Available: https://arxiv.org/abs/2601.15195

[3] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*. ACM, 2023, pp. 79–90.

[4] G. Guest, A. Bunce, and L. Johnson, "How many interviews are enough? an experiment with data saturation and variability," *Field Methods*, vol. 18, no. 1, pp. 59–82, 2006.

[5] Á. Hajdu, P. Gustafsson, T. Kapus, K. Mao, M. Marescotti, and D. Distefano, "Combining static and dynamic analysis to detect software defects," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2024, pp. 2209–2211.

[6] P. Hamfelt, R. Britto, L. S. Rocha, and C. C. Almendra, "Automatic identification of machine learning-specific code smells," in *Proceedings of the 39th Brazilian Symposium on Software Engineering (SBES)*, 2025, pp. 337–347.

[7] M. M. Hasan *et al.*, "Model context protocol (MCP) tool descriptions are smelly! towards improving AI agent efficiency with augmented MCP tool descriptions," 2026. [Online]. Available: https://arxiv.org/abs/2602.14878

[8] N. Islam, D. G. Thomas, S. Ahmed *et al.*, "When agents fail: A comprehensive study of bugs in LLM agents with automated labeling," in *Preprint*, 2026. [Online]. Available: https://arxiv.org/abs/2601.15232

[9] K. Ning, J. Chen, J. Zhang, W. Li, Z. Wang, Y. Feng, W. Zhang, and Z. Zheng, "Defining and detecting the defects of large language model-based autonomous agents," *IEEE Transactions on Software Engineering*, vol. 52, no. 3, pp. 1074–1093, 2026.

[10] P. Offermann, O. Levina, M. Schönherr, and U. Bub, "Outline of a design science research process," in *Proceedings of the 4th International Conference on Design Science Research in Information Systems and Technology*. ACM, 2009.

[11] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2008.

[12] Y. Shang, Y. Li, K. Zhao, L. Ma, J. Liu, F. Xu, and Y. Li, "AgentSquare: Automatic LLM agent search in modular design space," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://openreview.net/forum?id=mPdmDYIQ7f

[13] R. Souza *et al.*, "PROV-AGENT: Unified provenance for tracking AI agent interactions in agentic workflows," in *Proceedings of the 21st IEEE International Conference on e-Science (eScience)*, 2025, pp. 467–473.

[14] A. Strauss and J. Corbin, *Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory*, 3rd ed. Thousand Oaks, CA: Sage Publications, 2008.

[15] M. Suri *et al.*, "CodeScout: Contextual problem statement enhancement for software agents," 2026. [Online]. Available: https://arxiv.org/abs/2603.05744

[16] Traceloop, "OpenLLMetry: Open-source observability for GenAI applications based on OpenTelemetry," 2024, gitHub repository, accessed April 2026. [Online]. Available: https://github.com/traceloop/openllmetry

[17] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.

[18] D. Wu, F. Mu, L. Shi, Z. Guo, K. Liu, W. Zhuang, Y. Zhong, and L. Zhang, "iSMELL: Assembling LLMs with expert toolsets for code smell detection and refactoring," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2024.

[19] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Cheng, Q. Zhang, W. Qin, Y. Zheng, X. Qiu, X. Huang, and T. Gui, "The rise and potential of large language model based agents: A survey," *Science China Information Sciences*, vol. 68, no. 2, 2025.

[20] U. Yadav *et al.*, "SoK: Agentic retrieval-augmented generation (RAG): Taxonomy, architectures, evaluation, and research directions," 2026. [Online]. Available: https://arxiv.org/abs/2603.07379

[21] R. K. Yin, *Case Study Research and Applications: Design and Methods*, 6th ed. Thousand Oaks, CA: Sage Publications, 2018.

[22] H. Zhang, L. Cruz, and A. van Deursen, "Code smells for machine learning applications," in *Proceedings of the 1st International Conference on AI Engineering – Software Engineering for AI (CAIN)*. ACM/IEEE, 2022, pp. 217–228.

Appendix A

Developer Evaluation Questionnaire

This appendix reproduces the full developer evaluation questionnaire used in the qualitative evaluation (Section 3.1.5). The questionnaire was administered as an anonymous online form.

Introduction & Consent

AgentLint is a static and runtime analysis tool for detecting architectural smells in LLM-based agent systems. This evaluation is part of a bachelor thesis at Blekinge Institute of Technology (BTH) in collaboration with Ericsson. The evaluation takes approximately 15–20 minutes. Participation is voluntary and responses are anonymous.

Q0. I consent to participating in this evaluation and understand that my anonymous responses will be used for academic research. [Checkbox: Yes, I consent]

Background (Q1–Q4)

Q1. How many years of professional software development experience do you have? [Multiple choice: <1, 1–3, 3–5, 5–10, >10]

Q2. How familiar are you with LLM-based agent systems (e.g., LangChain, CrewAI, AutoGPT)? [Scale 1–5: No experience to Expert]

Q3. How familiar are you with static analysis / linting tools (e.g., pylint, flake8, ESLint)? [Scale 1–5: Never used to Use daily]

Q4. What is your current role? [Short text]

Hands-on Task (Q5–Q6)

Participants install AgentLint, run it against a provided sample project, and optionally apply it to their own code.

Q5. How many smells did AgentLint report on the sample project (static scan)? [Short text; expected: 35]

Q6. Did you also run AgentLint on your own agent code? [Yes / No]

Detection Quality (Q7–Q16)

Q7. The smell reports were easy to understand. [Scale 1–5]

Q8. The detected smells correspond to real problems I have seen in agent code. [Scale 1–5]

Q9. The detected smells correspond to problems I would expect to find in agent code. [Scale 1–5]

Q10. The severity ratings (low/medium/high) matched my intuition about the importance of each smell. [Scale 1–5]

Q11. The file locations and line numbers in the report were accurate enough to find the issue. [Scale 1–5]

Q12. The tool produced too many irrelevant warnings. [Scale 1–5]

Q13. Did you notice any false positives? [None / A few (1–2) / Several (3–5) / Many (>5)]

Q14. If you noticed false positives, which design rules were they from? [Checkboxes: DR-1 through DR-14, Not sure]

Q15. Did you notice any issues in the sample project that AgentLint did NOT flag? [No / Yes]

Q16. If AgentLint missed issues, please describe what it missed. [Long text]

Usefulness & Usability (Q17–Q22)

Q17. The tool provides a systematic way to identify architectural issues in agent code. [Scale 1–5]

Q18. I would use this tool in my development workflow. [Scale 1–5]

Q19. The CLI output format was clear and well-organized. [Scale 1–5]

Q20. The installation and setup process was straightforward. [Scale 1–5]

Q21. Which design rules felt MOST useful to you? [Checkboxes, select up to 3: DR-1 through DR-14]

Q22. Which design rules felt LEAST useful or too noisy? [Checkboxes: DR-1 through DR-14]

Perceived Impact (Q23–Q25)

Q23. How confident are you that fixing the detected smells would improve agent reliability? [Scale 1–5]

Q24. How confident are you that fixing the detected smells would reduce token consumption or API costs? [Scale 1–5]

Q25. Have you experienced any of these smell categories causing issues in production? [Checkboxes: Reasoning loops, Memory/context bloat, Persona/tool failures, Error recovery gaps, Prompt/instruction drift, State management issues, None]

Taxonomy Completeness (Q26–Q27)

Q26. Are there architectural problems in agent systems that you have encountered that are NOT covered by AgentLint's 14 design rules? [No / Yes]

Q27. If there are uncovered issues, please describe them. [Long text]

Open-Ended Feedback (Q28–Q30)

Q28. What did you find most valuable about AgentLint? [Long text]

Q29. What is missing from the tool that you would want to see? [Long text]

Q30. Any other comments or suggestions? [Long text]

Appendix B

Full Taxonomy of Design Rules

This appendix presents the complete taxonomy of 14 architectural design rules with descriptions, detection modes, and detection targets. Table 3.3 in Chapter 3 provides a summary; this appendix expands each rule for reference.

Table B.1: Full taxonomy of 14 architectural design rules

DR	Name	Category	Description	Mode	What It Detects
DR-1	Loop Budget	Reasoning Loops	Agent loops must have a configurable iteration ceiling	Static + Runtime	Loops around LLM calls without max-iteration guards
DR-2	Call Dedup	Reasoning Loops	Identical consecutive tool calls should be suppressed	Runtime only	Same tool called with identical arguments in sequence
DR-3	Context Window	Memory & Context	Message history growth must be bounded	Static + Runtime	Unbounded append in loops without size checks
DR-4	Tool Output Cap	Memory & Context	Tool output must be truncated before prompt insertion	Static + Runtime	Tool output concatenated without size checks
DR-5	Persona Pinning	Persona & Tool	System prompt must persist across tool-call boundaries	Static + Runtime	System parameter missing after tool calls
DR-6	Tool Error Contract	Persona & Tool	Tool calls must include error handling	Static only	Tool calls without try/except or error checking
DR-7	Parallel Isolation	State Mgmt	Shared resources must be synchronised across threads	Static + Runtime	Global mutables accessed without locks
DR-8	Bounded Retry	Error Recovery	Retries must include backoff or delay	Static + Runtime	Retry loops without sleep or backoff logic
DR-9	Fallback Path	Error Recovery	LLM calls must have a fallback for provider failure	Static + Runtime	Single-provider calls with no alternative path

DR	Name	Category	Description	Mode	What It Detects
DR-10	Input Isolation	Prompt & Instruction	User input must not be concatenated into system messages	Static + Runtime	User variables interpolated into system strings
DR-11	Immutable Templates	Prompt & Instruction	Prompt templates must not be reassigned after definition	Static + Runtime	Template strings overwritten or concatenated
DR-12	Synchronized State	State Mgmt	Shared state containers must use synchronisation	Static + Runtime	Shared dicts/lists accessed from multiple threads
DR-13	Serialization Safety	State Mgmt	State classes must implement serialisation	Static only	State classes without to_json/from_json methods
DR-14	Deterministic Order	State Mgmt	Task dispatch must use deterministic ordering	Static + Runtime	Iteration over sets or unordered collections

This appendix is referenced from Table 3.3 in Chapter 3.

Design Rule Examples and Consequences

The following provides a synthetic code example and consequence for each design rule.

DR-1 (Loop Budget)
`while not done: response = llm.chat(messages)` — no iteration limit. *Consequence:* infinite token consumption and potential cost explosion.

DR-2 (Call Dedup)
Agent calls `search("weather")` three times consecutively with identical arguments. *Consequence:* wasted API calls and tokens without new information.

DR-3 (Context Window)
`messages.append(response)` inside a loop without size guard. *Consequence:* context overflow causes truncation or API errors.

DR-4 (Tool Output Cap)
`messages.append(tool_result)` where tool returns unbounded text. *Consequence:* single tool response fills the context window.

DR-5 (Persona Pinning)
System prompt set at start but not re-injected after tool-call responses. *Consequence:* agent loses its role/persona mid-conversation.

DR-6 (Tool Error Contract)
`result = tool.invoke(query)` without try/except. *Consequence:* unhandled exception crashes the agent.

DR-7 (Parallel Isolation)
Multiple agents write to a shared `results = []` without locks. *Consequence:* race conditions corrupt shared state.

DR-8 (Bounded Retry)
`while True: try: llm.call() except: continue` — no backoff. *Consequence:* retry storm overwhelms the API; no recovery time.

DR-9 (Fallback Path)
Single `response = openai.chat()` with no alternative provider. *Consequence:* total failure if provider is down.

DR-10 (Input Isolation)
`prompt = f"You are a helper. User said: {user_input}"` — unsanitised. *Consequence:* prompt injection allows user to override system instructions.

DR-11 (Immutable Templates)
`template += extra_instructions` modifies the template at runtime. *Consequence:* instruction drift across turns; non-reproducible behaviour.

DR-12 (Synchronized State)
`agent_state["memory"] = new_data` from multiple async tasks. *Consequence:* concurrent writes corrupt agent memory.

DR-13 (Serialization Safety)
State class has no `to_json()` or `from_json()` method. *Consequence:* agent state cannot be persisted or recovered after crash.

DR-14 (Deterministic Order)
`for task in set(tasks): dispatch(task)` — unordered iteration. *Consequence:* non-reproducible execution order; debugging becomes impossible.



Faculty of Computing, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden